

# Two Pointer Templates

---

Two patterns — pick based on whether pointers converge or march together.

## Quick Reference Table

| #   | Name                                   | Description                                                                            | Intuition                                                                                                                               | Time     | Space                                        | Key Implementation |
|-----|----------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|----------|----------------------------------------------|--------------------|
| 167 | Two Sum II                             | Sorted array. Find two numbers summing to target. Return 1-based indices.              | Squeeze from both ends; each comparison tells you which side is wrong, so one move eliminates a whole row/column of pairs.              | $O(n)$   | $O(1)$                                       | Standard           |
| 15  | 3Sum                                   | Find all unique triplets summing to 0. No duplicate triplets in output.                | Pin one number, then it becomes Two Sum on the rest — turning a triple search into a sweep.                                             | $O(n^2)$ | $O(1)$ excluding output ( $O(n)$ sort stack) | Standard           |
| 18  | 4Sum                                   | Find all unique quadruplets summing to target.                                         | Pin two numbers, then the inner search is Two Sum — same idea as 3Sum with one more fixed level.                                        | $O(n^3)$ | $O(1)$ excluding output ( $O(n)$ sort stack) | Standard           |
| 11  | Container With Most Water              | Given heights of vertical lines, find two lines forming the container with most water. | Width shrinks every step, so only a taller boundary can ever help; abandon the shorter wall.                                            | $O(n)$   | $O(1)$                                       | Standard           |
| 42  | Trapping Rain Water                    | Given elevation heights, compute total water trapped after rain.                       | Water level at a cell is set by the shorter side's tallest wall, so always process from the lower side where that max is already known. | $O(n)$   | $O(1)$                                       | Standard           |
| 27  | Remove Element                         | Remove all occurrences of <code>val</code> in-place. Return new length.                | The writer only advances on keepers, so it always points at the next free slot for a kept value.                                        | $O(n)$   | $O(1)$                                       | Standard           |
| 26  | Remove Duplicates from Sorted Array    | Remove duplicates in-place so each element appears at most once. Return new length.    | Since the array is sorted, comparing against the last written value is enough to drop every duplicate run.                              | $O(n)$   | $O(1)$                                       | Standard           |
| 80  | Remove Duplicates from Sorted Array II | Each element may appear at most twice in-place. Return new length.                     | Looking two slots back lets at most two copies through before the third is rejected.                                                    | $O(n)$   | $O(1)$                                       | Standard           |
| 283 | Move Zeroes                            | Move all zeroes to end while preserving relative order of non-zero elements.           | Compact the non-zeros to the front in order, then pad the leftover tail with zeros.                                                     | $O(n)$   | $O(1)$                                       | Standard           |
| 75  | Sort Colors                            | Sort array of 0s, 1s, 2s in-place without library sort.                                | A single scan grows a "small" region from the left and a "large" region from the right, leaving mids in the middle.                     | $O(n)$   | $O(1)$                                       | Standard           |

| #   | Name                         | Description                                                                                                                    | Intuition                                                                                                                                            | Time           | Space                   | Key Implementation |
|-----|------------------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|-------------------------|--------------------|
| 1   | Two Sum                      | Given an integer array and a target sum, return the indices of two numbers that add up to target. Exactly one solution exists. | Trade space for time — remember every value you've passed so the complement is a single lookup.                                                      | $O(n)$         | $O(n)$                  | Standard           |
| 16  | 3Sum Closest                 | Find three integers whose sum is closest to target. Return that sum.                                                           | Same pin-and-sweep as 3Sum, but instead of matching exactly you keep the running closest.                                                            | $O(n^2)$       | $O(1)$                  | Standard           |
| 31  | Next Permutation             | Rearrange the array to its lexicographically next greater permutation in-place. If none, sort ascending.                       | The longest decreasing suffix is already maximal; bump the digit just before it up to the smallest larger value, then reset the suffix to ascending. | $O(n)$         | $O(1)$                  | Standard           |
| 41  | First Missing Positive       | Find the smallest missing positive integer in an unsorted array. $O(n)$ time, $O(1)$ space.                                    | Use the array itself as a hash table keyed by value, then scan for the first slot holding the wrong number.                                          | $O(n)$         | $O(1)$                  | Standard           |
| 48  | Rotate Image                 | Rotate an $n \times n$ matrix 90 degrees clockwise in-place.                                                                   | Transposing flips across the main diagonal; reversing each row then completes the clockwise turn.                                                    | $O(n^2)$       | $O(1)$                  | Standard           |
| 54  | Spiral Matrix                | Return all elements of an $m \times n$ matrix in spiral order.                                                                 | Peel the matrix like an onion, one boundary layer per loop.                                                                                          | $O(m \cdot n)$ | $O(1)$ excluding output | Standard           |
| 73  | Set Matrix Zeroes            | If any element is 0, set its entire row and column to 0, in-place.                                                             | Reuse the first row and column as the bookkeeping space so no extra arrays are needed.                                                               | $O(m \cdot n)$ | $O(1)$                  | Standard           |
| 128 | Longest Consecutive Sequence | Find the length of the longest run of consecutive integers in an unsorted array. $O(n)$ .                                      | Each sequence is counted exactly once by starting only at its smallest member.                                                                       | $O(n)$         | $O(n)$                  | Standard           |
| 164 | Maximum Gap                  | Find the maximum gap between successive elements in the sorted form of an array. $O(n)$ .                                      | With buckets sized at the minimum possible gap, the answer never lies inside a bucket, only between bucket boundaries.                               | $O(n)$         | $O(n)$                  | Standard           |
| 169 | Majority Element             | Find the element appearing more than $n/2$ times.                                                                              | Pairing off different elements cancels them out; the majority element always survives.                                                               | $O(n)$         | $O(1)$                  | Standard           |
| 189 | Rotate Array                 | Rotate the array right by $k$ positions, in-place.                                                                             | Two partial reversals after a full reversal place each block in its                                                                                  | $O(n)$         | $O(1)$                  | Standard           |

| #   | Name                           | Description                                                                                                                   | Intuition                                                                                                                          | Time                               | Space                           | Key Implementation                    |
|-----|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|---------------------------------|---------------------------------------|
|     |                                |                                                                                                                               | rotated position without extra space.                                                                                              |                                    |                                 |                                       |
| 217 | Contains Duplicate             | Return true if any value appears at least twice.                                                                              | A set rejects repeats, so the first failed insert proves a duplicate.                                                              | $O(n)$                             | $O(n)$                          | Standard                              |
| 220 | Contains Duplicate III         | Are there indices $i, j$ with $ \text{nums}[i] - \text{nums}[j]  \leq \text{valueDiff}$ and $ i - j  \leq \text{indexDiff}$ ? | Same-bucket means automatically within range; neighbor buckets need an explicit check. Slide a window of size $\text{indexDiff}$ . | $O(n)$                             | $O(\min(n, \text{indexDiff}))$  | Standard                              |
| 229 | Majority Element II            | Find all elements appearing more than $n/3$ times.                                                                            | At most two elements can each occur more than a third of the time, so track two running candidates.                                | $O(n)$                             | $O(1)$                          | Standard                              |
| 238 | Product of Array Except Self   | Return output where $\text{output}[i]$ is the product of all elements except $\text{nums}[i]$ . No division.                  | Each position's answer is everything to its left times everything to its right.                                                    | $O(n)$                             | $O(1)$ excluding output         | Standard                              |
| 259 | 3Sum Smaller                   | Count triplets with sum strictly less than target in a sorted array.                                                          | Once the largest partner works, every smaller partner does too, so count them in bulk.                                             | $O(n^2)$                           | $O(1)$                          | bulk-count all valid $j$ for this $i$ |
| 280 | Wiggle Sort                    | Reorder in-place so $\text{nums}[0] \leq \text{nums}[1] \geq \text{nums}[2] \leq \text{nums}[3] \dots$                        | A greedy local swap is always safe — it never breaks the pair you already fixed.                                                   | $O(n)$                             | $O(1)$                          | Standard                              |
| 289 | Game of Life                   | Simulate one Game of Life step, updating all cells simultaneously, in-place.                                                  | Pack the new state into a spare bit so neighbor counts still read the old state during the pass.                                   | $O(m \cdot n)$                     | $O(1)$                          | Standard                              |
| 303 | Range Sum Query - Immutable    | Answer many range-sum queries on a static array efficiently.                                                                  | A range sum is the difference of two prefix sums, so each query is $O(1)$ after one build pass.                                    | $O(n)$ build, $O(1)$ query         | $O(n)$                          | Standard                              |
| 304 | Range Sum Query 2D - Immutable | Answer many 2D region-sum queries on a static matrix efficiently.                                                             | Each region sum combines four corner prefix sums via inclusion-exclusion.                                                          | $O(m \cdot n)$ build, $O(1)$ query | $O(m \cdot n)$                  | Standard                              |
| 307 | Range Sum Query - Mutable      | Support range-sum queries and point updates.                                                                                  | A Fenwick tree stores partial sums over power-of-two ranges so updates and queries each touch only $\log n$ nodes.                 | $O(\log n)$ update/query           | $O(n)$                          | Standard                              |
| 311 | Sparse Matrix Multiplication   | Multiply two sparse matrices, skipping zero elements.                                                                         | A zero in $A$ contributes nothing, so skip it entirely instead of doing a full triple loop.                                        | $O(mkn)$ worst case                | $O(m \cdot n)$ excluding output | skip zero to exploit sparsity         |
| 315 | Count of Smaller Numbers After | For each element, count elements to its right that are smaller. $O(n \log n)$                                                 | During a merge, every time a right element is                                                                                      | $O(n \log n)$                      | $O(n)$                          | Standard                              |

| #   | Name                                     | Description                                                                                                        | Intuition                                                                                                             | Time                    | Space           | Key Implementation |
|-----|------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------|-----------------|--------------------|
|     | Self                                     | n).                                                                                                                | taken before a left one, it is a smaller-to-the-right for that left element.                                          |                         |                 |                    |
| 325 | Maximum Size Subarray Sum Equals k       | Find the longest subarray with sum equal to k. $O(n)$ .                                                            | A subarray sums to k exactly when two prefix sums differ by k; keep the earliest index to maximize length.            | $O(n)$                  | $O(n)$          | Standard           |
| 327 | Count of Range Sum                       | Count range sums that lie in [lower, upper]. $O(n \log n)$ .                                                       | A range sum is a difference of two prefix sums; merge sort lets you count qualifying pairs across halves efficiently. | $O(n \log n)$           | $O(n)$          | Standard           |
| 334 | Increasing Triplet Subsequence           | Return true if there exists $i < j < k$ with <code>nums[i] &lt; nums[j] &lt; nums[k]</code> .                      | Maintaining the two smallest ascending values seen so far is enough to detect a third larger one.                     | $O(n)$                  | $O(1)$          | Standard           |
| 349 | Intersection of Two Arrays               | Return the intersection of two arrays (unique elements only).                                                      | Set membership turns intersection into linear lookups.                                                                | $O(n + m)$              | $O(n)$          | Standard           |
| 350 | Intersection of Two Arrays II            | Return the intersection including duplicates (multiplicity is the min of counts).                                  | A frequency map lets each shared occurrence be matched at most once.                                                  | $O(n + m)$              | $O(\min(n, m))$ | Standard           |
| 370 | Range Addition                           | Apply range updates <code>[start, end] += inc</code> then return the final array.                                  | Mark only the endpoints of each update; a single prefix pass materializes all increments.                             | $O(n + \text{updates})$ | $O(n)$          | Standard           |
| 384 | Shuffle an Array                         | Return a uniformly random permutation of the array; support reset to original.                                     | Choosing each position's element uniformly from the remaining ones yields every permutation with equal probability.   | $O(n)$ per shuffle      | $O(n)$          | Standard           |
| 414 | Third Maximum Number                     | Return the third distinct maximum; if it doesn't exist, return the maximum.                                        | Keep a sorted top-three as you scan, skipping duplicates.                                                             | $O(n)$                  | $O(1)$          | Standard           |
| 419 | Battleships in a Board                   | Count battleships ('X' runs, horizontal or vertical, never adjacent).                                              | Each ship has exactly one cell with no ship-neighbor above or to its left, so count those.                            | $O(m \cdot n)$          | $O(1)$          | Standard           |
| 442 | Find All Duplicates in an Array          | Values in $[1, n]$ , each appears once or twice. Return the ones appearing twice. $O(n)$ time, $O(1)$ extra space. | Flip the sign at each value's home index; encountering an already-negative slot means that value repeated.            | $O(n)$                  | $O(1)$          | Standard           |
| 448 | Find All Numbers Disappeared in an Array | Values in $[1, n]$ . Return all numbers in $[1, n]$ missing from the array.                                        | Use sign marking; any home index never marked means that number was absent.                                           | $O(n)$                  | $O(1)$          | Standard           |

| #   | Name                                  | Description                                                                                                  | Intuition                                                                                                        | Time          | Space                     | Key Implementation |
|-----|---------------------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|---------------|---------------------------|--------------------|
| 454 | 4Sum II                               | Count quadruplets (i,j,k,l) with <code>A[i]+B[j]+C[k]+D[l] == 0</code> .                                     | Split four arrays into two pairs so a hash of one pair's sums turns the search into $O(n^2)$ lookups.            | $O(n^2)$      | $O(n^2)$                  | Standard           |
| 457 | Circular Array Loop                   | Detect a cycle of length > 1 with consistent direction in a circular array of jumps.                         | Fast/slow pointers detect cycles; reject single-element self-loops and direction flips.                          | $O(n)$        | $O(1)$                    | Standard           |
| 485 | Max Consecutive Ones                  | Find the maximum number of consecutive 1s in a binary array.                                                 | Grow a streak while seeing 1s and reset at each 0.                                                               | $O(n)$        | $O(1)$                    | Standard           |
| 493 | Reverse Pairs                         | Count pairs (i,j) with <code>i &lt; j</code> and <code>nums[i] &gt; 2 * nums[j]</code> .                     | Sorted halves let you count <code>nums[i] &gt; 2*nums[j]</code> pairs with a linear two-pointer sweep per merge. | $O(n \log n)$ | $O(n)$                    | Standard           |
| 498 | Diagonal Traverse                     | Traverse an $m \times n$ matrix diagonally, alternating direction each diagonal.                             | Cells on the same diagonal share <code>r+c</code> ; flip the read direction on alternating diagonals to zig-zag. | $O(m*n)$      | $O(1)$ excluding output   | Standard           |
| 517 | Super Washing Machines                | Minimize the max number of moves to equalize dresses across machines (one dress to a neighbor per move).     | A bottleneck is either a machine with too much surplus or a cut that must pass a large net flow.                 | $O(n)$        | $O(1)$                    | Standard           |
| 523 | Continuous Subarray Sum               | Is there a subarray of length $\geq 2$ whose sum is a multiple of $k$ ?                                      | Two prefix sums with the same remainder mod $k$ bound a subarray sum divisible by $k$ .                          | $O(n)$        | $O(\min(n, k))$           | Standard           |
| 525 | Contiguous Array                      | Find the longest subarray with equal numbers of 0s and 1s.                                                   | Equal counts means a running balance returns to a value seen before.                                             | $O(n)$        | $O(n)$                    | Standard           |
| 532 | K-diff Pairs in an Array              | Count unique pairs (i,j) where <code>nums[i] - nums[j] == k</code> ( $k \geq 0$ ).                           | Counting distinct values avoids duplicate pairs; the zero-diff case needs a repeated value.                      | $O(n)$        | $O(n)$                    | Standard           |
| 560 | Subarray Sum Equals K                 | Count subarrays summing to exactly $k$ .                                                                     | Each earlier prefix equal to <code>s-k</code> marks the start of a qualifying subarray ending here.              | $O(n)$        | $O(n)$                    | Standard           |
| 566 | Reshape the Matrix                    | Reshape an $m \times n$ matrix into $r \times c$ keeping row-major order; return original if sizes mismatch. | Both shapes share a linear index, so map between them with division and modulo.                                  | $O(m*n)$      | $O(r*c)$ excluding output | Standard           |
| 581 | Shortest Unsorted Continuous Subarray | Find the shortest subarray that, if sorted, makes the whole array sorted.                                    | The window starts where an element exceeds some later minimum and ends where an element falls                    | $O(n)$        | $O(1)$                    | Standard           |

| #   | Name                                     | Description                                                                                                                              | Intuition                                                                                                | Time                          | Space                           | Key Implementation            |
|-----|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-------------------------------|---------------------------------|-------------------------------|
|     |                                          |                                                                                                                                          | below some earlier maximum.                                                                              |                               |                                 |                               |
| 594 | Longest Harmonious Subsequence           | Find the longest subsequence where max and min differ by exactly 1.                                                                      | A harmonious subsequence uses exactly two adjacent values, so combine each pair's counts.                | $O(n)$                        | $O(n)$                          | Standard                      |
| 599 | Minimum Index Sum of Two Lists           | Find common strings between two lists with minimum index sum.                                                                            | Index sum is minimized greedily as you scan; ties are collected together.                                | $O(n + m)$                    | $O(n)$                          | Standard                      |
| 611 | Valid Triangle Number                    | Count triplets from an array of side lengths that form a valid triangle.                                                                 | Only the two smaller sides need to beat the largest, so fix the largest and bulk-count valid pairs.      | $O(n^2)$                      | $O(1)$                          | bulk-count valid i for this j |
| 661 | Image Smoother                           | Replace each cell with the floor of the average of itself and its up-to-8 neighbors.                                                     | Average over the in-bounds $3 \times 3$ window centered on each cell.                                    | $O(m \cdot n)$                | $O(m \cdot n)$ excluding output | Standard                      |
| 723 | Candy Crush                              | Repeatedly crush horizontal/vertical runs of 3+ same-positive candies and apply gravity, until stable.                                   | Mark all matches in one pass before clearing so simultaneous crushes are handled, then let candies fall. | $O((m \cdot n)^2)$ worst case | $O(1)$                          | Standard                      |
| 724 | Find Pivot Index                         | Find the leftmost index where the sum to its left equals the sum to its right.                                                           | Track a running left sum and derive the right sum from the precomputed total.                            | $O(n)$                        | $O(1)$                          | Standard                      |
| 766 | Toeplitz Matrix                          | Check if every top-left to bottom-right diagonal has a single value.                                                                     | Comparing each cell to its diagonal predecessor verifies all diagonals locally.                          | $O(m \cdot n)$                | $O(1)$                          | Standard                      |
| 769 | Max Chunks To Make Sorted                | Array is a permutation of $[0, n-1]$ . Max number of chunks that can be sorted independently to sort the whole.                          | When the largest value seen so far equals the index, everything up to here is a self-contained block.    | $O(n)$                        | $O(1)$                          | Standard                      |
| 795 | Number of Subarrays with Bounded Maximum | Count subarrays whose maximum lies in $[left, right]$ .                                                                                  | "Max in range" equals the difference of two "max at most X" counts, each computed in one pass.           | $O(n)$                        | $O(1)$                          | Standard                      |
| 807 | Max Increase to Keep City Skyline        | Increase building heights as much as possible without changing skylines viewed from any of 4 sides.                                      | A building is capped by the smaller of its row and column maxima to preserve both silhouettes.           | $O(n^2)$                      | $O(n)$                          | Standard                      |
| 825 | Friends Of Appropriate Ages              | Count friend requests; x friends y unless $age[y] \leq 0.5 \cdot age[x] + 7$ , $age[y] > age[x]$ , or $age[y] > 100$ && $age[x] < 100$ . | Ages are bounded, so counting per age-bucket pair collapses the problem to constant-size work.           | $O(n + 120^2)$                | $O(1)$                          | Standard                      |
| 838 | Push Dominoes                            | Simulate falling dominoes given a string of 'L', 'R', '.'.                                                                               | Each cell's outcome is the balance of forces                                                             | $O(n)$                        | $O(n)$                          | Standard                      |

| #    | Name                                            | Description                                                                     | Intuition                                                                                                 | Time                     | Space                   | Key Implementation |
|------|-------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------|-------------------------|--------------------|
|      |                                                 |                                                                                 | from the nearest pushers on either side.                                                                  |                          |                         |                    |
| 845  | Longest Mountain in Array                       | Find the length of the longest mountain (strictly up then strictly down).       | Scan to a peak by rising, then descend; the span counts only when both directions occurred.               | $O(n)$                   | $O(1)$                  | Standard           |
| 896  | Monotonic Array                                 | Return true if the array is entirely non-increasing or entirely non-decreasing. | A single pass noting direction changes detects non-monotonicity.                                          | $O(n)$                   | $O(1)$                  | Standard           |
| 912  | Sort an Array                                   | Sort an integer array. Implement an $O(n \log n)$ sort.                         | Recursively split, sort halves, and merge — stable $O(n \log n)$ without library sort.                    | $O(n \log n)$            | $O(n)$                  | Standard           |
| 974  | Subarray Sums Divisible by K                    | Count subarrays whose sum is divisible by k.                                    | Two prefixes with the same remainder mod k bound a divisible subarray; count pairs per remainder.         | $O(n)$                   | $O(k)$                  | Standard           |
| 977  | Squares of a Sorted Array                       | Return the squares of a sorted array in sorted order.                           | The biggest squares sit at the extremes, so fill the result from the largest slot inward.                 | $O(n)$                   | $O(1)$ excluding output | Standard           |
| 1013 | Partition Array Into Three Parts With Equal Sum | Can the array be split into three contiguous parts with equal sums?             | Greedily close off a part each time the running sum hits one-third; succeed if at least three parts form. | $O(n)$                   | $O(1)$                  | Standard           |
| 1074 | Number of Submatrices That Sum to Target        | Count submatrices summing to target.                                            | Reduce 2D to 1D by fixing the top and bottom rows, then count target-sum subarrays.                       | $O(m^2 * n)$             | $O(n)$                  | Standard           |
| 1109 | Corporate Flight Bookings                       | Given bookings [first, last, seats], return total seats booked per flight.      | Range increments become two endpoint marks resolved by one prefix pass.                                   | $O(n + \text{bookings})$ | $O(n)$                  | Standard           |
| 1213 | Intersection of Three Sorted Arrays             | Return the common elements of three sorted arrays.                              | Like merging — advance whichever pointer lags so all three meet at shared values.                         | $O(n)$                   | $O(1)$ excluding output | Standard           |
| 1275 | Find Winner on a Tic Tac Toe Game               | Given alternating A/B moves, determine the winner, "Draw", or "Pending".        | Signed counts per line reveal a win the moment any line reaches +3 or -3.                                 | $O(1)$                   | $O(1)$                  | Standard           |
| 1351 | Count Negative Numbers in a Sorted Matrix       | Count negatives in a matrix sorted descending by row and column.                | From the bottom-left, sorted order lets each step rule out a whole row or column of negatives.            | $O(m + n)$               | $O(1)$                  | Standard           |
| 1424 | Diagonal Traverse II                            | Traverse a jagged 2D list diagonally from bottom-left to top-right.             | Cells on a diagonal share $r + c$ ; emitting rows in descending order gives the                           | $O(\text{total})$        | $O(\text{total})$       | Standard           |



| #    | Name                                                        | Description                                                                               | Intuition                                                                                           | Time          | Space                   | Key Implementation |
|------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|---------------|-------------------------|--------------------|
|      |                                                             |                                                                                           | bottom-left-up direction.                                                                           |               |                         |                    |
| 1460 | Make Two Arrays Equal by Reversing Sub-arrays               | Can target become arr after reversing any sub-arrays any number of times?                 | Any permutation is reachable through reversals, so only element counts matter.                      | $O(n)$        | $O(1)$                  | Standard           |
| 1480 | Running Sum of 1d Array                                     | Return the running (prefix) sum of the array.                                             | Each running sum is the prior running sum plus the current value.                                   | $O(n)$        | $O(1)$                  | Standard           |
| 1498 | Number of Subsequences That Satisfy the Given Sum Condition | Count subsequences where $\min + \max \leq \text{target}$ . Answer mod $1e9+7$ .          | After sorting, fixing the minimum lets every subset of the elements up to the max be chosen freely. | $O(n \log n)$ | $O(n)$                  | Standard           |
| 1748 | Sum of Unique Elements                                      | Sum all elements that appear exactly once.                                                | Count occurrences, then sum only the singletons.                                                    | $O(n)$        | $O(1)$                  | Standard           |
| 1762 | Buildings With an Ocean View                                | Return indices of buildings with nothing taller to their right (ocean to the right).      | Sweeping from the ocean side, only record buildings that exceed every taller one already passed.    | $O(n)$        | $O(1)$ excluding output | Standard           |
| 1868 | Product of Two Run-Length Encoded Arrays                    | Multiply two run-length-encoded arrays element-wise; return the RLE product.              | March through both run lists together, consuming the shorter overlap and coalescing equal results.  | $O(n + m)$    | $O(1)$ excluding output | Standard           |
| 1877 | Minimize Maximum Pair Sum in Array                          | Pair up elements to minimize the maximum pair sum.                                        | Pairing extremes balances the sums, keeping the largest pair as small as possible.                  | $O(n \log n)$ | $O(1)$                  | Standard           |
| 1893 | Check if All the Integers in a Range Are Covered            | Are all integers in $[\text{left}, \text{right}]$ covered by at least one given interval? | Mark interval starts and ends, then a prefix sweep shows which points are covered.                  | $O(n + 50)$   | $O(1)$                  | Standard           |
| 1894 | Find the Student that Will Replace the Chalk                | Students consume chalk in order cyclically; find who runs out.                            | Whole cycles repeat, so only the remainder after a full round determines the failing student.       | $O(n)$        | $O(1)$                  | Standard           |
| 1920 | Build Array from Permutation                                | Return <code>result[i] = nums[nums[i]]</code> .                                           | Each output is a double lookup into the permutation.                                                | $O(n)$        | $O(n)$ excluding output | Standard           |
| 1929 | Concatenation of Array                                      | Return <code>nums</code> concatenated with itself.                                        | Copy each element into position <code>i</code> and <code>i+n</code> .                               | $O(n)$        | $O(1)$ excluding output | Standard           |

## Pattern 1 — Opposite Two Pointers

```
// MENTAL MODEL: two ends close in on the answer; each comparison rules out one whole end of the range.
// WHY sorted: moving inward only safely eliminates half the space if the array is sorted
int i = 0, j = n - 1;
while (i < j) {
    if (condition == target) {
        /* record */ i++; j--;
    } else if (tooSmall) {
        i++;
    } else {
        j--;
    }
}
```

Sort first. `i` and `j` converge inward. Used when the array is sorted and you can eliminate half the search space each step.

## #167 Two Sum II

**Description:** Sorted array. Find two numbers summing to target. Return 1-based indices.

**Key:** sum too small → move `i` right. sum too large → move `j` left.

**Intuition:** Squeeze from both ends; each comparison tells you which side is wrong, so one move eliminates a whole row/column of pairs.

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        int i = 0, j = nums.length - 1;
        while (i < j) {
            int sum = nums[i] + nums[j];
            if (sum == target) {
                return new int[]{i + 1, j + 1};
            } else if (sum < target) {
                i++;
            } else {
                j--;
            }
        }
        return new int[]{-1, -1};
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## #15 3Sum

**Description:** Find all unique triplets summing to 0. No duplicate triplets in output.

**Key:** sort first, fix `a`, run opposite two pointers on `[a+1, n-1]`. Skip duplicates at all three levels.

**Intuition:** Pin one number, then it becomes Two Sum on the rest — turning a triple search into a sweep.

```

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        for (int a = 0; a < nums.length - 2; a++) {
            if (a > 0 && nums[a] == nums[a - 1]) continue; // skip dup a
            int i = a + 1, j = nums.length - 1;
            while (i < j) {
                int sum = nums[a] + nums[i] + nums[j];
                if (sum == 0) {
                    result.add(Arrays.asList(nums[a], nums[i++], nums[j--]));
                    // dup-skip after a hit - see "Duplicate-skip Cheat Sheet" below
                    while (i < j && nums[i] == nums[i - 1]) i++; // skip dup i
                    while (i < j && nums[j] == nums[j + 1]) j--; // skip dup j
                } else if (sum < 0) {
                    i++;
                } else {
                    j--;
                }
            }
        }
        return result;
    }
}

```

**Time**  $O(n^2)$  | **Space**  $O(1)$  excluding output ( $O(n)$  sort stack)

## #18 4Sum

**Description:** Find all unique quadruplets summing to target.

**Key:** two nested fix loops + same opposite two pointer inner loop. Cast to `long` — sum of 4 ints can overflow.

**Intuition:** Pin two numbers, then the inner search is Two Sum — same idea as 3Sum with one more fixed level.

```

class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        int n = nums.length;
        for (int a = 0; a < n - 3; a++) {
            if (a > 0 && nums[a] == nums[a - 1]) continue;
            for (int b = a + 1; b < n - 2; b++) {
                if (b > a + 1 && nums[b] == nums[b - 1]) continue;
                int i = b + 1, j = n - 1;
                while (i < j) {
                    long sum = (long) nums[a] + nums[b] + nums[i] + nums[j];
                    if (sum == target) {
                        result.add(Arrays.asList(nums[a], nums[b], nums[i++], nums[j--]));
                        // dup-skip after a hit - see "Duplicate-skip Cheat Sheet" below
                        while (i < j && nums[i] == nums[i - 1]) i++;
                        while (i < j && nums[j] == nums[j + 1]) j--;
                    } else if (sum < target) {
                        i++;
                    } else {
                        j--;
                    }
                }
            }
        }
        return result;
    }
}

```

**Time**  $O(n^3)$  | **Space**  $O(1)$  excluding output ( $O(n)$  sort stack)

## #11 Container With Most Water

**Description:** Given heights of vertical lines, find two lines forming the container with most water.

**Key:** always move the shorter side inward — moving the taller side can only make things worse.

**Intuition:** Width shrinks every step, so only a taller boundary can ever help; abandon the shorter wall.

```

class Solution {
    public int maxArea(int[] height) {
        int i = 0, j = height.length - 1, max = 0;
        while (i < j) {
            max = Math.max(max, Math.min(height[i], height[j]) * (j - i));
            if (height[i] < height[j]) {
                i++;
            } else {
                j--;
            }
        }
        return max;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #42 Trapping Rain Water

**Description:** Given elevation heights, compute total water trapped after rain.

**Key:** water above position  $x = \min(\text{maxLeft}, \text{maxRight}) - \text{height}[x]$ . Track running max from each side.

**Intuition:** Water level at a cell is set by the shorter side's tallest wall, so always process from the lower side where that max is already known.

```
class Solution {
    public int trap(int[] height) {
        int i = 0, j = height.length - 1, water = 0;
        int maxI = 0, maxJ = 0;
        while (i < j) {
            if (height[i] <= height[j]) {
                maxI = Math.max(maxI, height[i]);
                water += maxI - height[i];
                i++;
            } else {
                maxJ = Math.max(maxJ, height[j]);
                water += maxJ - height[j];
                j--;
            }
        }
        return water;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## Pattern 2 — Same Direction (Write Pointer)

```
// a fast reader scans everything; a slow writer trails behind and only copies the keepers. - WHEN: fi
int i = 0; // i = write position
for (int j = 0; j < n; j++) { // j = read position
    if (keepCondition(nums[j]))
        nums[i++] = nums[j];
}
return i;
```

$i$  marks where the next valid element goes.  $j$  scans every element. Only write when the element passes the keep condition.

The generalised duplicate-skip pattern for "at most  $k$  duplicates":

```
int i = 0;
for (int j = 0; j < n; j++)
    if (i < k || nums[j] != nums[i - k]) // compare write head to k slots back
        nums[i++] = nums[j];
return i;
```

- $k = 1 \rightarrow$  #26 (no duplicates)
- $k = 2 \rightarrow$  #80 (at most 2)

## #27 Remove Element

**Description:** Remove all occurrences of `val` in-place. Return new length.

**Keep condition:** `nums[j] != val`

**Intuition:** The writer only advances on keepers, so it always points at the next free slot for a kept value.

```
class Solution {
    public int removeElement(int[] nums, int val) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (nums[j] != val) nums[i++] = nums[j];
        return i;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## #26 Remove Duplicates from Sorted Array

**Description:** Remove duplicates in-place so each element appears at most once. Return new length.

**Keep condition:** `nums[j] != nums[i-1]` — don't write if same as last written.

**Intuition:** Since the array is sorted, comparing against the last written value is enough to drop every duplicate run.

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (i == 0 || nums[j] != nums[i - 1])
                nums[i++] = nums[j];
        return i;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## #80 Remove Duplicates from Sorted Array II

**Description:** Each element may appear at most twice in-place. Return new length.

**Keep condition:** `nums[j] != nums[i-2]` — don't write if same as two slots back.

**Intuition:** Looking two slots back lets at most two copies through before the third is rejected.

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (i < 2 || nums[j] != nums[i - 2])
                nums[i++] = nums[j];
        return i;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## #283 Move Zeroes

**Description:** Move all zeroes to end while preserving relative order of non-zero elements.

**Keep condition:** `nums[j] != 0`. Fill tail with zeros after the write pass.

**Intuition:** Compact the non-zeros to the front in order, then pad the leftover tail with zeros.

```
class Solution {
    public void moveZeroes(int[] nums) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (nums[j] != 0) nums[i++] = nums[j];
        while (i < nums.length) nums[i++] = 0;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## #75 Sort Colors

**Description:** Sort array of 0s, 1s, 2s in-place without library sort.

**Key:** Dutch Flag with LO = 0, MID = 1, HI = 2.

**Intuition:** A single scan grows a "small" region from the left and a "large" region from the right, leaving mids in the middle.

```
class Solution {
    public void sortColors(int[] nums) {
        int i = 0, k = 0, j = nums.length - 1;
        while (k <= j) {
            if (nums[k] == 0) {
                swap(nums, i++, k++);
            } else if (nums[k] == 1) {
                k++;
            } else {
                swap(nums, k, j--);
            }
        }
    }
    private void swap(int[] a, int x, int y) {
        int t = a[x]; a[x] = a[y]; a[y] = t;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## Pattern Comparison

| Pattern        | Pointers move                                                                   | Sort needed   | Use when                                                                   |
|----------------|---------------------------------------------------------------------------------|---------------|----------------------------------------------------------------------------|
| Opposite       | <code>i</code> right, <code>j</code> left, converge                             | Yes (usually) | Sum/target problems on sorted array                                        |
| Same direction | <code>j</code> scans, <code>i</code> writes                                     | No            | Filter / compact array in-place                                            |
| Dutch Flag     | <code>k</code> scans, <code>i</code> low boundary, <code>j</code> high boundary | No            | 3-way partition ( <code>&lt;</code> / <code>=</code> / <code>&gt;</code> ) |

## Duplicate-skip Cheat Sheet (for Opposite pointers after a hit)

```
// After recording a result at (i, j):
i++; j--;
while (i < j && nums[i] == nums[i - 1]) i++; // skip duplicate i
while (i < j && nums[j] == nums[j + 1]) j--; // skip duplicate j
```

Always check `i < j` inside the skip loop to avoid crossing.

# Additional Reference Problems

## #1 Two Sum

**Description:** Given an integer array and a target sum, return the indices of two numbers that add up to target. Exactly one solution exists.

**Key:** hash map from value to index; for each element check if `target - x` was seen.

**Intuition:** Trade space for time — remember every value you've passed so the complement is a single lookup.

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> seen = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (seen.containsKey(need)) {
                return new int[]{seen.get(need), i};
            }
            seen.put(nums[i], i);
        }
        return new int[]{-1, -1};
    }
}
```

**Time**  $O(n)$  | **Space**  $O(n)$

## #16 3Sum Closest

**Description:** Find three integers whose sum is closest to target. Return that sum.

**Key:** sort, fix one, opposite two pointers; track the closest sum seen.

**Intuition:** Same pin-and-sweep as 3Sum, but instead of matching exactly you keep the running closest.

```
class Solution {
    public int threeSumClosest(int[] nums, int target) {
        Arrays.sort(nums);
        int best = nums[0] + nums[1] + nums[2];
        for (int a = 0; a < nums.length - 2; a++) {
            int i = a + 1, j = nums.length - 1;
            while (i < j) {
                int sum = nums[a] + nums[i] + nums[j];
                if (Math.abs(sum - target) < Math.abs(best - target)) {
                    best = sum;
                }
                if (sum == target) {
                    return sum;
                } else if (sum < target) {
                    i++;
                } else {
                    j--;
                }
            }
        }
        return best;
    }
}
```

**Time**  $O(n^2)$  | **Space**  $O(1)$



---

## #31 Next Permutation

**Description:** Rearrange the array to its lexicographically next greater permutation in-place. If none, sort ascending.

**Key:** find first descending pivot from the right, swap with next larger, reverse the suffix.

**Intuition:** The longest decreasing suffix is already maximal; bump the digit just before it up to the smallest larger value, then reset the suffix to ascending.

```
class Solution {
    public void nextPermutation(int[] nums) {
        int n = nums.length;
        int i = n - 2;
        while (i >= 0 && nums[i] >= nums[i + 1]) {
            i--;
        }
        if (i >= 0) {
            int j = n - 1;
            while (nums[j] <= nums[i]) {
                j--;
            }
            swap(nums, i, j);
        }
        reverse(nums, i + 1, n - 1);
    }
    private void reverse(int[] a, int x, int y) {
        while (x < y) {
            swap(a, x++, y--);
        }
    }
    private void swap(int[] a, int x, int y) {
        int t = a[x]; a[x] = a[y]; a[y] = t;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

---

## #41 First Missing Positive

**Description:** Find the smallest missing positive integer in an unsorted array.  $O(n)$  time,  $O(1)$  space.

**Key:** place each value `v` in `[1, n]` at index `v-1` via swaps; first index where `nums[i] != i+1` is the answer.

**Intuition:** Use the array itself as a hash table keyed by value, then scan for the first slot holding the wrong number.

```

class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] != nums[i]) {
                swap(nums, i, nums[i] - 1);
            }
        }
        for (int i = 0; i < n; i++) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        return n + 1;
    }
    private void swap(int[] a, int x, int y) {
        int t = a[x]; a[x] = a[y]; a[y] = t;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #48 Rotate Image

**Description:** Rotate an  $n \times n$  matrix 90 degrees clockwise in-place.

**Key:** transpose, then reverse each row.

**Intuition:** Transposing flips across the main diagonal; reversing each row then completes the clockwise turn.

```

class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int t = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = t;
            }
        }
        for (int i = 0; i < n; i++) {
            int lo = 0, hi = n - 1;
            while (lo < hi) {
                int t = matrix[i][lo];
                matrix[i][lo] = matrix[i][hi];
                matrix[i][hi] = t;
                lo++;
                hi--;
            }
        }
    }
}

```

**Time**  $O(n^2)$  | **Space**  $O(1)$

## #54 Spiral Matrix

**Description:** Return all elements of an  $m \times n$  matrix in spiral order.

**Key:** maintain four shrinking boundaries (top, bottom, left, right); walk each edge then close in.

**Intuition:** Peel the matrix like an onion, one boundary layer per loop.

```
class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        List<Integer> result = new ArrayList<>();
        int top = 0, bottom = matrix.length - 1;
        int left = 0, right = matrix[0].length - 1;
        while (top <= bottom && left <= right) {
            for (int j = left; j <= right; j++) {
                result.add(matrix[top][j]);
            }
            top++;
            for (int i = top; i <= bottom; i++) {
                result.add(matrix[i][right]);
            }
            right--;
            if (top <= bottom) {
                for (int j = right; j >= left; j--) {
                    result.add(matrix[bottom][j]);
                }
                bottom--;
            }
            if (left <= right) {
                for (int i = bottom; i >= top; i--) {
                    result.add(matrix[i][left]);
                }
                left++;
            }
        }
        return result;
    }
}
```

**Time**  $O(m*n)$  | **Space**  $O(1)$  excluding output

## #73 Set Matrix Zeroes

**Description:** If any element is 0, set its entire row and column to 0, in-place.

**Key:** use first row/column as marker storage; handle their own zero state with two flags.

**Intuition:** Reuse the first row and column as the bookkeeping space so no extra arrays are needed.

```

class Solution {
    public void setZeroes(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        boolean firstRow = false, firstCol = false;
        for (int j = 0; j < n; j++) {
            if (matrix[0][j] == 0) {
                firstRow = true;
            }
        }
        for (int i = 0; i < m; i++) {
            if (matrix[i][0] == 0) {
                firstCol = true;
            }
        }
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (matrix[i][j] == 0) {
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }
            }
        }
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                    matrix[i][j] = 0;
                }
            }
        }
        if (firstRow) {
            for (int j = 0; j < n; j++) {
                matrix[0][j] = 0;
            }
        }
        if (firstCol) {
            for (int i = 0; i < m; i++) {
                matrix[i][0] = 0;
            }
        }
    }
}

```

**Time**  $O(m \cdot n)$  | **Space**  $O(1)$

## #128 Longest Consecutive Sequence

**Description:** Find the length of the longest run of consecutive integers in an unsorted array.  $O(n)$ .

**Key:** put all in a set; only start counting from numbers with no predecessor (  $x-1$  absent).

**Intuition:** Each sequence is counted exactly once by starting only at its smallest member.

```

class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int x : nums) {
            set.add(x);
        }
        int best = 0;
        for (int x : set) {
            if (!set.contains(x - 1)) {
                int cur = x, len = 1;
                while (set.contains(cur + 1)) {
                    cur++;
                    len++;
                }
                best = Math.max(best, len);
            }
        }
        return best;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(n)$

## #164 Maximum Gap

**Description:** Find the maximum gap between successive elements in the sorted form of an array.  $O(n)$ .

**Key:** bucket sort — the max gap is at least  $\text{ceil}((\text{max}-\text{min})/(\text{n}-1))$ ; compare across adjacent non-empty buckets.

**Intuition:** With buckets sized at the minimum possible gap, the answer never lies inside a bucket, only between bucket boundaries.

```

class Solution {
    public int maximumGap(int[] nums) {
        int n = nums.length;
        if (n < 2) {
            return 0;
        }
        int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
        for (int x : nums) {
            min = Math.min(min, x);
            max = Math.max(max, x);
        }
        if (min == max) {
            return 0;
        }
        int bucketSize = Math.max(1, (max - min) / (n - 1));
        int bucketCount = (max - min) / bucketSize + 1;
        int[] bucketMin = new int[bucketCount];
        int[] bucketMax = new int[bucketCount];
        Arrays.fill(bucketMin, Integer.MAX_VALUE);
        Arrays.fill(bucketMax, Integer.MIN_VALUE);
        for (int x : nums) {
            int b = (x - min) / bucketSize;
            bucketMin[b] = Math.min(bucketMin[b], x);
            bucketMax[b] = Math.max(bucketMax[b], x);
        }
        int gap = 0, prevMax = min;
        for (int b = 0; b < bucketCount; b++) {
            if (bucketMin[b] == Integer.MAX_VALUE) {
                continue;
            }
            gap = Math.max(gap, bucketMin[b] - prevMax);
            prevMax = bucketMax[b];
        }
        return gap;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(n)$

## #169 Majority Element

**Description:** Find the element appearing more than  $n/2$  times.

**Key:** Boyer-Moore voting — keep a candidate and a count; flip candidate when count hits 0.

**Intuition:** Pairing off different elements cancels them out; the majority element always survives.

```

class Solution {
    public int majorityElement(int[] nums) {
        int candidate = nums[0], count = 0;
        for (int x : nums) {
            if (count == 0) {
                candidate = x;
            }
            if (x == candidate) {
                count++;
            } else {
                count--;
            }
        }
        return candidate;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #189 Rotate Array

**Description:** Rotate the array right by k positions, in-place.

**Key:** reverse whole array, then reverse first k and last n-k.

**Intuition:** Two partial reversals after a full reversal place each block in its rotated position without extra space.

```

class Solution {
    public void rotate(int[] nums, int k) {
        int n = nums.length;
        k %= n;
        reverse(nums, 0, n - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, n - 1);
    }
    private void reverse(int[] a, int i, int j) {
        while (i < j) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++;
            j--;
        }
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #217 Contains Duplicate

**Description:** Return true if any value appears at least twice.

**Key:** add to a set; if an insert fails, a duplicate exists.

**Intuition:** A set rejects repeats, so the first failed insert proves a duplicate.

```

class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();
        for (int x : nums) {
            if (!seen.add(x)) {
                return true;
            }
        }
        return false;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(n)$

## #220 Contains Duplicate III

**Description:** Are there indices  $i, j$  with  $|nums[i] - nums[j]| \leq \text{valueDiff}$  and  $|i - j| \leq \text{indexDiff}$ ?

**Key:** bucket by value width ( $\text{valueDiff} + 1$ ); only adjacent buckets can satisfy the value constraint.

**Intuition:** Same-bucket means automatically within range; neighbor buckets need an explicit check. Slide a window of size  $\text{indexDiff}$ .

```

class Solution {
    public boolean containsNearbyAlmostDuplicate(int[] nums, int indexDiff, int valueDiff) {
        if (valueDiff < 0) {
            return false;
        }
        Map<Long, Long> buckets = new HashMap<>();
        long width = (long) valueDiff + 1;
        for (int i = 0; i < nums.length; i++) {
            long id = bucketId(nums[i], width);
            if (buckets.containsKey(id)) {
                return true;
            }
            if (buckets.containsKey(id - 1) && Math.abs(nums[i] - buckets.get(id - 1)) < width) {
                return true;
            }
            if (buckets.containsKey(id + 1) && Math.abs(nums[i] - buckets.get(id + 1)) < width) {
                return true;
            }
            buckets.put(id, (long) nums[i]);
            if (i >= indexDiff) {
                buckets.remove(bucketId(nums[i - indexDiff], width));
            }
        }
        return false;
    }
    private long bucketId(long x, long width) {
        return x < 0 ? (x + 1) / width - 1 : x / width;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(\min(n, \text{indexDiff}))$

## #229 Majority Element II

**Description:** Find all elements appearing more than  $n/3$  times.

**Key:** Boyer-Moore with two candidates (at most two can exceed  $n/3$ ); verify with a second pass.



**Intuition:** At most two elements can each occur more than a third of the time, so track two running candidates.

```
class Solution {
    public List<Integer> majorityElement(int[] nums) {
        int c1 = 0, c2 = 1, n1 = 0, n2 = 0;
        for (int x : nums) {
            if (x == c1) {
                n1++;
            } else if (x == c2) {
                n2++;
            } else if (n1 == 0) {
                c1 = x;
                n1 = 1;
            } else if (n2 == 0) {
                c2 = x;
                n2 = 1;
            } else {
                n1--;
                n2--;
            }
        }
        n1 = 0;
        n2 = 0;
        for (int x : nums) {
            if (x == c1) {
                n1++;
            } else if (x == c2) {
                n2++;
            }
        }
        List<Integer> result = new ArrayList<>();
        if (n1 > nums.length / 3) {
            result.add(c1);
        }
        if (n2 > nums.length / 3) {
            result.add(c2);
        }
        return result;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## #238 Product of Array Except Self

**Description:** Return output where `output[i]` is the product of all elements except `nums[i]`. No division.

**Key:** prefix products left-to-right, then multiply by suffix products right-to-left.

**Intuition:** Each position's answer is everything to its left times everything to its right.

```

class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        result[0] = 1;
        for (int i = 1; i < n; i++) {
            result[i] = result[i - 1] * nums[i - 1];
        }
        int suffix = 1;
        for (int i = n - 1; i >= 0; i--) {
            result[i] *= suffix;
            suffix *= nums[i];
        }
        return result;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$  excluding output

## #259 3Sum Smaller

**Description:** Count triplets with sum strictly less than target in a sorted array.

**Key:** fix one, opposite pointers; when `sum < target`, all pairs between `i` and `j` also qualify → add `j-i`.

**Intuition:** Once the largest partner works, every smaller partner does too, so count them in bulk.

```

class Solution {
    public int threeSumSmaller(int[] nums, int target) {
        Arrays.sort(nums);
        int count = 0;
        for (int a = 0; a < nums.length - 2; a++) {
            int i = a + 1, j = nums.length - 1;
            while (i < j) {
                if (nums[a] + nums[i] + nums[j] < target) {
                    count += j - i; // ← VARIATION: bulk-count all valid j for this i
                    i++;
                } else {
                    j--;
                }
            }
        }
        return count;
    }
}

```

**Time**  $O(n^2)$  | **Space**  $O(1)$

## #280 Wiggle Sort

**Description:** Reorder in-place so `nums[0] <= nums[1] >= nums[2] <= nums[3]...`

**Key:** single pass; fix each adjacent pair that violates the expected up/down relation by swapping.

**Intuition:** A greedy local swap is always safe — it never breaks the pair you already fixed.

```

class Solution {
    public void wiggleSort(int[] nums) {
        for (int i = 0; i < nums.length - 1; i++) {
            boolean shouldRise = (i % 2 == 0);
            if (shouldRise == (nums[i] > nums[i + 1])) {
                int t = nums[i]; nums[i] = nums[i + 1]; nums[i + 1] = t;
            }
        }
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #289 Game of Life

**Description:** Simulate one Game of Life step, updating all cells simultaneously, in-place.

**Key:** encode next state in the second bit (e.g. 01 → 11 for live→live); shift right at the end.

**Intuition:** Pack the new state into a spare bit so neighbor counts still read the old state during the pass.

```

class Solution {
    public void gameOfLife(int[][] board) {
        int m = board.length, n = board[0].length;
        int[] dr = {1, -1, 0, 0, 1, 1, -1, -1};
        int[] dc = {0, 0, 1, -1, 1, -1, 1, -1};
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                int live = 0;
                for (int dir = 0; dir < 8; dir++) {
                    int nr = r + dr[dir], nc = c + dc[dir];
                    if (nr >= 0 && nr < m && nc >= 0 && nc < n && (board[nr][nc] & 1) == 1) {
                        live++;
                    }
                }
                if ((board[r][c] & 1) == 1) {
                    if (live == 2 || live == 3) {
                        board[r][c] |= 2;
                    }
                } else {
                    if (live == 3) {
                        board[r][c] |= 2;
                    }
                }
            }
        }
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                board[r][c] >>= 1;
            }
        }
    }
}

```

**Time**  $O(m*n)$  | **Space**  $O(1)$

## #303 Range Sum Query - Immutable

**Description:** Answer many range-sum queries on a static array efficiently.

**Key:** precompute prefix sums; `sumRange(i,j) = prefix[j+1] - prefix[i]` .

**Intuition:** A range sum is the difference of two prefix sums, so each query is  $O(1)$  after one build pass.

```
class NumArray {
    private int[] prefix;
    public NumArray(int[] nums) {
        prefix = new int[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }
    public int sumRange(int i, int j) {
        return prefix[j + 1] - prefix[i];
    }
}
```

**Time**  $O(n)$  build,  $O(1)$  query | **Space**  $O(n)$

## #304 Range Sum Query 2D - Immutable

**Description:** Answer many 2D region-sum queries on a static matrix efficiently.

**Key:** 2D prefix sums with inclusion-exclusion.

**Intuition:** Each region sum combines four corner prefix sums via inclusion-exclusion.

```
class NumMatrix {
    private int[][] prefix;
    public NumMatrix(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        prefix = new int[m + 1][n + 1];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                prefix[i + 1][j + 1] = matrix[i][j] + prefix[i][j + 1]
                    + prefix[i + 1][j] - prefix[i][j];
            }
        }
    }
    public int sumRegion(int row1, int col1, int row2, int col2) {
        return prefix[row2 + 1][col2 + 1] - prefix[row1][col2 + 1]
            - prefix[row2 + 1][col1] + prefix[row1][col1];
    }
}
```

**Time**  $O(mn)$  build,  $O(1)$  query | **Space**  $O(mn)$

## #307 Range Sum Query - Mutable

**Description:** Support range-sum queries and point updates.

**Key:** Binary Indexed Tree (Fenwick) — both update and prefix query in  $O(\log n)$ .

**Intuition:** A Fenwick tree stores partial sums over power-of-two ranges so updates and queries each touch only  $\log n$  nodes.

```

class NumArray {
    private int[] tree;
    private int[] nums;
    private int n;
    public NumArray(int[] nums) {
        this.n = nums.length;
        this.nums = new int[n];
        this.tree = new int[n + 1];
        for (int i = 0; i < n; i++) {
            update(i, nums[i]);
        }
    }
    public void update(int index, int val) {
        int delta = val - nums[index];
        nums[index] = val;
        for (int i = index + 1; i <= n; i += i & (-i)) {
            tree[i] += delta;
        }
    }
    public int sumRange(int left, int right) {
        return prefix(right + 1) - prefix(left);
    }
    private int prefix(int i) {
        int s = 0;
        while (i > 0) {
            s += tree[i];
            i -= i & (-i);
        }
        return s;
    }
}

```

**Time**  $O(\log n)$  update/query | **Space**  $O(n)$

## #311 Sparse Matrix Multiplication

**Description:** Multiply two sparse matrices, skipping zero elements.

**Key:** iterate only over non-zero entries of the first matrix; for each, fan out across the matching row of the second.

**Intuition:** A zero in A contributes nothing, so skip it entirely instead of doing a full triple loop.

```

class Solution {
    public int[][] multiply(int[][] mat1, int[][] mat2) {
        int m = mat1.length, k = mat1[0].length, n = mat2[0].length;
        int[][] result = new int[m][n];
        for (int i = 0; i < m; i++) {
            for (int p = 0; p < k; p++) {
                if (mat1[i][p] == 0) {
                    continue; // ← VARIATION: skip zero to exploit sparsity
                }
                for (int j = 0; j < n; j++) {
                    if (mat2[p][j] != 0) {
                        result[i][j] += mat1[i][p] * mat2[p][j];
                    }
                }
            }
        }
        return result;
    }
}

```

**Time**  $O(mkn)$  worst case | **Space**  $O(m \cdot n)$  excluding output

## #315 Count of Smaller Numbers After Self

**Description:** For each element, count elements to its right that are smaller.  $O(n \log n)$ .

**Key:** merge sort on (value, index) pairs; while merging, count how many right-half elements slot before a left element.

**Intuition:** During a merge, every time a right element is taken before a left one, it is a smaller-to-the-right for that left element.

```
class Solution {
    private int[] counts;
    public List<Integer> countSmaller(int[] nums) {
        int n = nums.length;
        counts = new int[n];
        int[] indices = new int[n];
        for (int i = 0; i < n; i++) {
            indices[i] = i;
        }
        mergeSort(nums, indices, 0, n - 1);
        List<Integer> result = new ArrayList<>();
        for (int c : counts) {
            result.add(c);
        }
        return result;
    }
    private void mergeSort(int[] nums, int[] idx, int lo, int hi) {
        if (lo >= hi) {
            return;
        }
        int mid = (lo + hi) >> 1;
        mergeSort(nums, idx, lo, mid);
        mergeSort(nums, idx, mid + 1, hi);
        int[] merged = new int[hi - lo + 1];
        int i = lo, j = mid + 1, k = 0, rightCount = 0;
        while (i <= mid && j <= hi) {
            if (nums[idx[j]] < nums[idx[i]]) {
                rightCount++;
                merged[k++] = idx[j++];
            } else {
                counts[idx[i]] += rightCount;
                merged[k++] = idx[i++];
            }
        }
        while (i <= mid) {
            counts[idx[i]] += rightCount;
            merged[k++] = idx[i++];
        }
        while (j <= hi) {
            merged[k++] = idx[j++];
        }
        System.arraycopy(merged, 0, idx, lo, merged.length);
    }
}
```

**Time**  $O(n \log n)$  | **Space**  $O(n)$

## #325 Maximum Size Subarray Sum Equals k

---

**Description:** Find the longest subarray with sum equal to k.  $O(n)$ .

**Key:** prefix sum + hash map of earliest index for each prefix; for current sum  $s$ , look for  $s - k$ .

**Intuition:** A subarray sums to k exactly when two prefix sums differ by k; keep the earliest index to maximize length.

```
class Solution {
    public int maxSubArrayLen(int[] nums, int k) {
        Map<Integer, Integer> firstIndex = new HashMap<>();
        firstIndex.put(0, -1);
        int sum = 0, best = 0;
        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
            if (firstIndex.containsKey(sum - k)) {
                best = Math.max(best, i - firstIndex.get(sum - k));
            }
            firstIndex.putIfAbsent(sum, i);
        }
        return best;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(n)$

---

## #327 Count of Range Sum

---

**Description:** Count range sums that lie in [lower, upper].  $O(n \log n)$ .

**Key:** prefix sums + merge sort; while merging count prefix pairs whose difference falls in range.

**Intuition:** A range sum is a difference of two prefix sums; merge sort lets you count qualifying pairs across halves efficiently.

```

class Solution {
    private long lower, upper;
    public int countRangeSum(int[] nums, int lower, int upper) {
        this.lower = lower;
        this.upper = upper;
        long[] prefix = new long[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
        return mergeCount(prefix, 0, prefix.length - 1);
    }
    private int mergeCount(long[] prefix, int lo, int hi) {
        if (lo >= hi) {
            return 0;
        }
        int mid = (lo + hi) >> 1;
        int count = mergeCount(prefix, lo, mid) + mergeCount(prefix, mid + 1, hi);
        int low = mid + 1, high = mid + 1;
        for (int i = lo; i <= mid; i++) {
            while (low <= hi && prefix[low] - prefix[i] < lower) {
                low++;
            }
            while (high <= hi && prefix[high] - prefix[i] <= upper) {
                high++;
            }
            count += high - low;
        }
        long[] merged = new long[hi - lo + 1];
        int i = lo, j = mid + 1, k = 0;
        while (i <= mid && j <= hi) {
            if (prefix[i] <= prefix[j]) {
                merged[k++] = prefix[i++];
            } else {
                merged[k++] = prefix[j++];
            }
        }
        while (i <= mid) {
            merged[k++] = prefix[i++];
        }
        while (j <= hi) {
            merged[k++] = prefix[j++];
        }
        System.arraycopy(merged, 0, prefix, lo, merged.length);
        return count;
    }
}

```

**Time**  $O(n \log n)$  | **Space**  $O(n)$

## #334 Increasing Triplet Subsequence

**Description:** Return true if there exists  $i < j < k$  with  $nums[i] < nums[j] < nums[k]$ .

**Key:** track smallest ( first ) and smallest second value ( second ); any element beating second completes the triplet.

**Intuition:** Maintaining the two smallest ascending values seen so far is enough to detect a third larger one.



```

class Solution {
    public boolean increasingTriplet(int[] nums) {
        int first = Integer.MAX_VALUE, second = Integer.MAX_VALUE;
        for (int x : nums) {
            if (x <= first) {
                first = x;
            } else if (x <= second) {
                second = x;
            } else {
                return true;
            }
        }
        return false;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #349 Intersection of Two Arrays

**Description:** Return the intersection of two arrays (unique elements only).

**Key:** put one array in a set; collect elements of the other that are present.

**Intuition:** Set membership turns intersection into linear lookups.

```

class Solution {
    public int[] intersection(int[] nums1, int[] nums2) {
        Set<Integer> set = new HashSet<>();
        for (int x : nums1) {
            set.add(x);
        }
        Set<Integer> common = new HashSet<>();
        for (int x : nums2) {
            if (set.contains(x)) {
                common.add(x);
            }
        }
        int[] result = new int[common.size()];
        int i = 0;
        for (int x : common) {
            result[i++] = x;
        }
        return result;
    }
}

```

**Time**  $O(n + m)$  | **Space**  $O(n)$

## #350 Intersection of Two Arrays II

**Description:** Return the intersection including duplicates (multiplicity is the min of counts).

**Key:** count one array in a map; for the other, emit and decrement while count > 0.

**Intuition:** A frequency map lets each shared occurrence be matched at most once.

```

class Solution {
    public int[] intersect(int[] nums1, int[] nums2) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int x : nums1) {
            count.merge(x, 1, Integer::sum);
        }
        List<Integer> result = new ArrayList<>();
        for (int x : nums2) {
            if (count.getOrDefault(x, 0) > 0) {
                result.add(x);
                count.put(x, count.get(x) - 1);
            }
        }
        int[] arr = new int[result.size()];
        for (int i = 0; i < arr.length; i++) {
            arr[i] = result.get(i);
        }
        return arr;
    }
}

```

**Time**  $O(n + m)$  | **Space**  $O(\min(n, m))$

## #370 Range Addition

**Description:** Apply range updates `[start, end] += inc` then return the final array.

**Key:** difference array — add `inc` at `start`, subtract at `end+1`, then prefix-sum.

**Intuition:** Mark only the endpoints of each update; a single prefix pass materializes all increments.

```

class Solution {
    public int[] getModifiedArray(int length, int[][] updates) {
        int[] diff = new int[length + 1];
        for (int[] u : updates) {
            diff[u[0]] += u[2];
            diff[u[1] + 1] -= u[2];
        }
        int[] result = new int[length];
        int running = 0;
        for (int i = 0; i < length; i++) {
            running += diff[i];
            result[i] = running;
        }
        return result;
    }
}

```

**Time**  $O(n + \text{updates})$  | **Space**  $O(n)$

## #384 Shuffle an Array

**Description:** Return a uniformly random permutation of the array; support reset to original.

**Key:** Fisher-Yates — for each `i` swap with a random index in `[i, n)`.

**Intuition:** Choosing each position's element uniformly from the remaining ones yields every permutation with equal probability.

```

class Solution {
    private int[] original;
    private int[] arr;
    private Random rng = new Random();
    public Solution(int[] nums) {
        original = nums.clone();
        arr = nums.clone();
    }
    public int[] reset() {
        arr = original.clone();
        return arr;
    }
    public int[] shuffle() {
        for (int i = arr.length - 1; i > 0; i--) {
            int j = rng.nextInt(i + 1);
            int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
        }
        return arr;
    }
}

```

**Time**  $O(n)$  per shuffle | **Space**  $O(n)$

## #414 Third Maximum Number

**Description:** Return the third distinct maximum; if it doesn't exist, return the maximum.

**Key:** track three distinct maxima with Long sentinels to handle Integer.MIN\_VALUE.

**Intuition:** Keep a sorted top-three as you scan, skipping duplicates.

```

class Solution {
    public int thirdMax(int[] nums) {
        Long first = null, second = null, third = null;
        for (int x : nums) {
            Long v = (long) x;
            if (v.equals(first) || v.equals(second) || v.equals(third)) {
                continue;
            }
            if (first == null || v > first) {
                third = second;
                second = first;
                first = v;
            } else if (second == null || v > second) {
                third = second;
                second = v;
            } else if (third == null || v > third) {
                third = v;
            }
        }
        return (int) (long) (third == null ? first : third);
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #419 Battleships in a Board

**Description:** Count battleships ('X' runs, horizontal or vertical, never adjacent).

**Key:** count only the top-left cell of each ship — an 'X' with no 'X' directly above or left.

**Intuition:** Each ship has exactly one cell with no ship-neighbor above or to its left, so count those.

```
class Solution {
    public int countBattleships(char[][] board) {
        int count = 0;
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[0].length; c++) {
                if (board[r][c] == 'X'
                    && (r == 0 || board[r - 1][c] != 'X')
                    && (c == 0 || board[r][c - 1] != 'X')) {
                    count++;
                }
            }
        }
        return count;
    }
}
```

**Time**  $O(m \cdot n)$  | **Space**  $O(1)$

## #442 Find All Duplicates in an Array

**Description:** Values in  $[1, n]$ , each appears once or twice. Return the ones appearing twice.  $O(n)$  time,  $O(1)$  extra space.

**Key:** use the sign at index  $|x|-1$  as a "seen" marker; a second visit finds it already negative.

**Intuition:** Flip the sign at each value's home index; encountering an already-negative slot means that value repeated.

```
class Solution {
    public List<Integer> findDuplicates(int[] nums) {
        List<Integer> result = new ArrayList<>();
        for (int x : nums) {
            int idx = Math.abs(x) - 1;
            if (nums[idx] < 0) {
                result.add(idx + 1);
            } else {
                nums[idx] = -nums[idx];
            }
        }
        return result;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## #448 Find All Numbers Disappeared in an Array

**Description:** Values in  $[1, n]$ . Return all numbers in  $[1, n]$  missing from the array.

**Key:** mark index  $|x|-1$  negative; positive slots correspond to missing numbers.

**Intuition:** Use sign marking; any home index never marked means that number was absent.

```

class Solution {
    public List<Integer> findDisappearedNumbers(int[] nums) {
        for (int x : nums) {
            int idx = Math.abs(x) - 1;
            if (nums[idx] > 0) {
                nums[idx] = -nums[idx];
            }
        }
        List<Integer> result = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] > 0) {
                result.add(i + 1);
            }
        }
        return result;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #454 4Sum II

**Description:** Count quadruplets  $(i,j,k,l)$  with  $A[i]+B[j]+C[k]+D[l] == 0$ .

**Key:** map all sums of  $A+B$ ; for each  $C+D$  look up  $-(c+d)$ .

**Intuition:** Split four arrays into two pairs so a hash of one pair's sums turns the search into  $O(n^2)$  lookups.

```

class Solution {
    public int fourSumCount(int[] a, int[] b, int[] c, int[] d) {
        Map<Integer, Integer> abSum = new HashMap<>();
        for (int x : a) {
            for (int y : b) {
                abSum.merge(x + y, 1, Integer::sum);
            }
        }
        int count = 0;
        for (int x : c) {
            for (int y : d) {
                count += abSum.getOrDefault(-(x + y), 0);
            }
        }
        return count;
    }
}

```

**Time**  $O(n^2)$  | **Space**  $O(n^2)$

## #457 Circular Array Loop

**Description:** Detect a cycle of length  $> 1$  with consistent direction in a circular array of jumps.

**Key:** Floyd's tortoise and hare; movement direction must stay consistent and the cycle length must exceed 1.

**Intuition:** Fast/slow pointers detect cycles; reject single-element self-loops and direction flips.

```

class Solution {
    private int n;
    public boolean circularArrayLoop(int[] nums) {
        n = nums.length;
        for (int i = 0; i < n; i++) {
            int slow = i, fast = i;
            boolean forward = nums[i] > 0;
            do {
                slow = next(nums, slow, forward);
                if (slow == -1) {
                    break;
                }
                fast = next(nums, fast, forward);
                if (fast == -1) {
                    break;
                }
                fast = next(nums, fast, forward);
                if (fast == -1) {
                    break;
                }
            } while (slow != fast);
            if (slow != -1 && slow == fast) {
                return true;
            }
        }
        return false;
    }
    private int next(int[] nums, int i, boolean forward) {
        if ((nums[i] > 0) != forward) {
            return -1;
        }
        int j = ((i + nums[i]) % n + n) % n;
        if (j == i) {
            return -1;
        }
        return j;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #485 Max Consecutive Ones

**Description:** Find the maximum number of consecutive 1s in a binary array.

**Key:** running counter, reset on 0, track the max.

**Intuition:** Grow a streak while seeing 1s and reset at each 0.

```

class Solution {
    public int findMaxConsecutiveOnes(int[] nums) {
        int best = 0, cur = 0;
        for (int x : nums) {
            if (x == 1) {
                cur++;
                best = Math.max(best, cur);
            } else {
                cur = 0;
            }
        }
        return best;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #493 Reverse Pairs

**Description:** Count pairs  $(i, j)$  with  $i < j$  and  $nums[i] > 2 * nums[j]$ .

**Key:** merge sort; before merging each half, count reverse pairs across the sorted halves.

**Intuition:** Sorted halves let you count  $nums[i] > 2 * nums[j]$  pairs with a linear two-pointer sweep per merge.

```

class Solution {
    public int reversePairs(int[] nums) {
        return mergeSort(nums, 0, nums.length - 1);
    }
    private int mergeSort(int[] nums, int lo, int hi) {
        if (lo >= hi) {
            return 0;
        }
        int mid = (lo + hi) >> 1;
        int count = mergeSort(nums, lo, mid) + mergeSort(nums, mid + 1, hi);
        int j = mid + 1;
        for (int i = lo; i <= mid; i++) {
            while (j <= hi && (long) nums[i] > 2L * nums[j]) {
                j++;
            }
            count += j - (mid + 1);
        }
        int[] merged = new int[hi - lo + 1];
        int i = lo, p = mid + 1, k = 0;
        while (i <= mid && p <= hi) {
            if (nums[i] <= nums[p]) {
                merged[k++] = nums[i++];
            } else {
                merged[k++] = nums[p++];
            }
        }
        while (i <= mid) {
            merged[k++] = nums[i++];
        }
        while (p <= hi) {
            merged[k++] = nums[p++];
        }
        System.arraycopy(merged, 0, nums, lo, merged.length);
        return count;
    }
}

```

**Time**  $O(n \log n)$  | **Space**  $O(n)$

## #498 Diagonal Traverse

**Description:** Traverse an  $m \times n$  matrix diagonally, alternating direction each diagonal.

**Key:** group by  $r+c$  ; even diagonals go up-right, odd go down-left.

**Intuition:** Cells on the same diagonal share  $r+c$  ; flip the read direction on alternating diagonals to zig-zag.

```
class Solution {
    public int[] findDiagonalOrder(int[][] mat) {
        int m = mat.length, n = mat[0].length;
        int[] result = new int[m * n];
        int idx = 0, r = 0, c = 0;
        boolean up = true;
        while (idx < m * n) {
            result[idx++] = mat[r][c];
            if (up) {
                if (c == n - 1) {
                    r++;
                    up = false;
                } else if (r == 0) {
                    c++;
                    up = false;
                } else {
                    r--;
                    c++;
                }
            } else {
                if (r == m - 1) {
                    c++;
                    up = true;
                } else if (c == 0) {
                    r++;
                    up = true;
                } else {
                    r++;
                    c--;
                }
            }
        }
        return result;
    }
}
```

**Time**  $O(m*n)$  | **Space**  $O(1)$  excluding output

## #517 Super Washing Machines

**Description:** Minimize the max number of moves to equalize dresses across machines (one dress to a neighbor per move).

**Key:** if total not divisible by  $n$ , impossible; answer is max of  $|\text{prefixImbalance}|$  and any single overflow  $\text{load} - \text{avg}$ .

**Intuition:** A bottleneck is either a machine with too much surplus or a cut that must pass a large net flow.



```

class Solution {
    public int findMinMoves(int[] machines) {
        int total = 0;
        for (int m : machines) {
            total += m;
        }
        int n = machines.length;
        if (total % n != 0) {
            return -1;
        }
        int avg = total / n;
        int result = 0, running = 0;
        for (int m : machines) {
            int diff = m - avg;
            running += diff;
            result = Math.max(result, Math.max(Math.abs(running), diff));
        }
        return result;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #523 Continuous Subarray Sum

**Description:** Is there a subarray of length  $\geq 2$  whose sum is a multiple of  $k$ ?

**Key:** store earliest index for each prefix-sum mod  $k$ ; equal remainders far apart mean a divisible subarray.

**Intuition:** Two prefix sums with the same remainder mod  $k$  bound a subarray sum divisible by  $k$ .

```

class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
        Map<Integer, Integer> firstIndex = new HashMap<>();
        firstIndex.put(0, -1);
        int sum = 0;
        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
            int rem = ((sum % k) + k) % k;
            if (firstIndex.containsKey(rem)) {
                if (i - firstIndex.get(rem) >= 2) {
                    return true;
                }
            } else {
                firstIndex.put(rem, i);
            }
        }
        return false;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(\min(n, k))$

## #525 Contiguous Array

**Description:** Find the longest subarray with equal numbers of 0s and 1s.

**Key:** treat 0 as -1; longest subarray with sum 0 via prefix-sum first-index map.

**Intuition:** Equal counts means a running balance returns to a value seen before.

```

class Solution {
    public int findMaxLength(int[] nums) {
        Map<Integer, Integer> firstIndex = new HashMap<>();
        firstIndex.put(0, -1);
        int balance = 0, best = 0;
        for (int i = 0; i < nums.length; i++) {
            balance += (nums[i] == 1) ? 1 : -1;
            if (firstIndex.containsKey(balance)) {
                best = Math.max(best, i - firstIndex.get(balance));
            } else {
                firstIndex.put(balance, i);
            }
        }
        return best;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(n)$

## #532 K-diff Pairs in an Array

**Description:** Count unique pairs  $(i, j)$  where  $\text{nums}[i] - \text{nums}[j] == k$  ( $k \geq 0$ ).

**Key:** frequency map; for  $k=0$  count values with freq  $\geq 2$ , else count distinct values  $v$  with  $v+k$  present.

**Intuition:** Counting distinct values avoids duplicate pairs; the zero-diff case needs a repeated value.

```

class Solution {
    public int findPairs(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int x : nums) {
            count.merge(x, 1, Integer::sum);
        }
        int result = 0;
        for (Map.Entry<Integer, Integer> e : count.entrySet()) {
            if (k == 0) {
                if (e.getValue() >= 2) {
                    result++;
                }
            } else if (count.containsKey(e.getKey() + k)) {
                result++;
            }
        }
        return result;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(n)$

## #560 Subarray Sum Equals K

**Description:** Count subarrays summing to exactly  $k$ .

**Key:** prefix-sum frequency map; for current sum  $s$  add count of  $s - k$  seen.

**Intuition:** Each earlier prefix equal to  $s-k$  marks the start of a qualifying subarray ending here.

```

class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        count.put(0, 1);
        int sum = 0, result = 0;
        for (int x : nums) {
            sum += x;
            result += count.getOrDefault(sum - k, 0);
            count.merge(sum, 1, Integer::sum);
        }
        return result;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(n)$

## #566 Reshape the Matrix

**Description:** Reshape an  $m \times n$  matrix into  $r \times c$  keeping row-major order; return original if sizes mismatch.

**Key:** flatten index  $t$ ; source  $(t/n, t\%n)$ , target  $(t/c, t\%c)$ .

**Intuition:** Both shapes share a linear index, so map between them with division and modulo.

```

class Solution {
    public int[][] matrixReshape(int[][] mat, int r, int c) {
        int m = mat.length, n = mat[0].length;
        if (m * n != r * c) {
            return mat;
        }
        int[][] result = new int[r][c];
        for (int t = 0; t < m * n; t++) {
            result[t / c][t % c] = mat[t / n][t % n];
        }
        return result;
    }
}

```

**Time**  $O(mn)$  | **Space**  $O(rc)$  excluding output

## #581 Shortest Unsorted Continuous Subarray

**Description:** Find the shortest subarray that, if sorted, makes the whole array sorted.

**Key:** scan from right tracking min to find left bound; scan from left tracking max to find right bound.

**Intuition:** The window starts where an element exceeds some later minimum and ends where an element falls below some earlier maximum.

```

class Solution {
    public int findUnsortedSubarray(int[] nums) {
        int n = nums.length;
        int left = -1, right = -2;
        int max = nums[0], min = nums[n - 1];
        for (int i = 1; i < n; i++) {
            max = Math.max(max, nums[i]);
            if (nums[i] < max) {
                right = i;
            }
        }
        for (int i = n - 2; i >= 0; i--) {
            min = Math.min(min, nums[i]);
            if (nums[i] > min) {
                left = i;
            }
        }
        return right - left + 1;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #594 Longest Harmonious Subsequence

**Description:** Find the longest subsequence where max and min differ by exactly 1.

**Key:** frequency map; for each value  $v$ , if  $v+1$  exists, candidate length is `count[v] + count[v+1]`.

**Intuition:** A harmonious subsequence uses exactly two adjacent values, so combine each pair's counts.

```

class Solution {
    public int findLHS(int[] nums) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int x : nums) {
            count.merge(x, 1, Integer::sum);
        }
        int best = 0;
        for (Map.Entry<Integer, Integer> e : count.entrySet()) {
            if (count.containsKey(e.getKey() + 1)) {
                best = Math.max(best, e.getValue() + count.get(e.getKey() + 1));
            }
        }
        return best;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(n)$

## #599 Minimum Index Sum of Two Lists

**Description:** Find common strings between two lists with minimum index sum.

**Key:** map first list value to index; scan second tracking the minimum index sum.

**Intuition:** Index sum is minimized greedily as you scan; ties are collected together.

```

class Solution {
    public String[] findRestaurant(String[] list1, String[] list2) {
        Map<String, Integer> index = new HashMap<>();
        for (int i = 0; i < list1.length; i++) {
            index.put(list1[i], i);
        }
        List<String> result = new ArrayList<>();
        int best = Integer.MAX_VALUE;
        for (int j = 0; j < list2.length; j++) {
            if (index.containsKey(list2[j])) {
                int sum = j + index.get(list2[j]);
                if (sum < best) {
                    best = sum;
                    result.clear();
                    result.add(list2[j]);
                } else if (sum == best) {
                    result.add(list2[j]);
                }
            }
        }
        return result.toArray(new String[0]);
    }
}

```

**Time**  $O(n + m)$  | **Space**  $O(n)$

## #611 Valid Triangle Number

**Description:** Count triplets from an array of side lengths that form a valid triangle.

**Key:** sort; fix the largest side  $k$ , opposite pointers on  $[0, k-1]$ ; if  $\text{nums}[i] + \text{nums}[j] > \text{nums}[k]$ , all  $i..j$  qualify.

**Intuition:** Only the two smaller sides need to beat the largest, so fix the largest and bulk-count valid pairs.

```

class Solution {
    public int triangleNumber(int[] nums) {
        Arrays.sort(nums);
        int count = 0;
        for (int k = nums.length - 1; k >= 2; k--) {
            int i = 0, j = k - 1;
            while (i < j) {
                if (nums[i] + nums[j] > nums[k]) {
                    count += j - i;    // ← VARIATION: bulk-count valid i for this j
                    j--;
                } else {
                    i++;
                }
            }
        }
        return count;
    }
}

```

**Time**  $O(n^2)$  | **Space**  $O(1)$

## #661 Image Smoother

**Description:** Replace each cell with the floor of the average of itself and its up-to-8 neighbors.

**Key:** for each cell sum valid neighbors (including self) and divide by the count.

**Intuition:** Average over the in-bounds 3×3 window centered on each cell.

```
class Solution {
    public int[][] imageSmoother(int[][] img) {
        int m = img.length, n = img[0].length;
        int[][] result = new int[m][n];
        int[] dr = {1, -1, 0, 0, 1, 1, -1, -1, 0};
        int[] dc = {0, 0, 1, -1, 1, -1, 1, -1, 0};
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                int sum = 0, cnt = 0;
                for (int dir = 0; dir < 9; dir++) {
                    int nr = r + dr[dir], nc = c + dc[dir];
                    if (nr >= 0 && nr < m && nc >= 0 && nc < n) {
                        sum += img[nr][nc];
                        cnt++;
                    }
                }
                result[r][c] = sum / cnt;
            }
        }
        return result;
    }
}
```

**Time**  $O(mn)$  | **Space**  $O(mn)$  excluding output

## #723 Candy Crush

**Description:** Repeatedly crush horizontal/vertical runs of 3+ same-positive candies and apply gravity, until stable.

**Key:** mark crushable cells (negate), zero them, drop columns; repeat until no change.

**Intuition:** Mark all matches in one pass before clearing so simultaneous crushes are handled, then let candies fall.

```

class Solution {
    public int[][] candyCrush(int[][] board) {
        int m = board.length, n = board[0].length;
        boolean changed = true;
        while (changed) {
            changed = false;
            for (int r = 0; r < m; r++) {
                for (int c = 0; c + 2 < n; c++) {
                    int v = Math.abs(board[r][c]);
                    if (v != 0 && v == Math.abs(board[r][c + 1]) && v == Math.abs(board[r][c + 2])) {
                        board[r][c] = board[r][c + 1] = board[r][c + 2] = -v;
                        changed = true;
                    }
                }
            }
            for (int r = 0; r + 2 < m; r++) {
                for (int c = 0; c < n; c++) {
                    int v = Math.abs(board[r][c]);
                    if (v != 0 && v == Math.abs(board[r + 1][c]) && v == Math.abs(board[r + 2][c])) {
                        board[r][c] = board[r + 1][c] = board[r + 2][c] = -v;
                        changed = true;
                    }
                }
            }
            if (changed) {
                for (int c = 0; c < n; c++) {
                    int write = m - 1;
                    for (int r = m - 1; r >= 0; r--) {
                        if (board[r][c] > 0) {
                            board[write--][c] = board[r][c];
                        }
                    }
                    while (write >= 0) {
                        board[write--][c] = 0;
                    }
                }
            }
        }
        return board;
    }
}

```

**Time**  $O((m*n)^2)$  worst case | **Space**  $O(1)$

## #724 Find Pivot Index

**Description:** Find the leftmost index where the sum to its left equals the sum to its right.

**Key:** total sum minus left and current gives right sum; check `left == total - left - nums[i]`.

**Intuition:** Track a running left sum and derive the right sum from the precomputed total.

```

class Solution {
    public int pivotIndex(int[] nums) {
        int total = 0;
        for (int x : nums) {
            total += x;
        }
        int left = 0;
        for (int i = 0; i < nums.length; i++) {
            if (left == total - left - nums[i]) {
                return i;
            }
            left += nums[i];
        }
        return -1;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #766 Toeplitz Matrix

**Description:** Check if every top-left to bottom-right diagonal has a single value.

**Key:** each cell (except first row/col) must equal its up-left neighbor.

**Intuition:** Comparing each cell to its diagonal predecessor verifies all diagonals locally.

```

class Solution {
    public boolean isToeplitzMatrix(int[][] matrix) {
        for (int r = 1; r < matrix.length; r++) {
            for (int c = 1; c < matrix[0].length; c++) {
                if (matrix[r][c] != matrix[r - 1][c - 1]) {
                    return false;
                }
            }
        }
        return true;
    }
}

```

**Time**  $O(m*n)$  | **Space**  $O(1)$

## #769 Max Chunks To Make Sorted

**Description:** Array is a permutation of  $[0, n-1]$ . Max number of chunks that can be sorted independently to sort the whole.

**Key:** a chunk boundary occurs wherever the running max equals the current index.

**Intuition:** When the largest value seen so far equals the index, everything up to here is a self-contained block.



```

class Solution {
    public int maxChunksToSorted(int[] arr) {
        int max = 0, chunks = 0;
        for (int i = 0; i < arr.length; i++) {
            max = Math.max(max, arr[i]);
            if (max == i) {
                chunks++;
            }
        }
        return chunks;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #795 Number of Subarrays with Bounded Maximum

**Description:** Count subarrays whose maximum lies in  $[left, right]$ .

**Key:** count subarrays with  $\max \leq right$  minus those with  $\max \leq left-1$ .

**Intuition:** "Max in range" equals the difference of two "max at most X" counts, each computed in one pass.

```

class Solution {
    public int numSubarrayBoundedMax(int[] nums, int left, int right) {
        return countMaxAtMost(nums, right) - countMaxAtMost(nums, left - 1);
    }
    private int countMaxAtMost(int[] nums, int bound) {
        int count = 0, run = 0;
        for (int x : nums) {
            if (x <= bound) {
                run++;
            } else {
                run = 0;
            }
            count += run;
        }
        return count;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #807 Max Increase to Keep City Skyline

**Description:** Increase building heights as much as possible without changing skylines viewed from any of 4 sides.

**Key:** each cell can rise to  $\min(rowMax, colMax)$  ; sum the increases.

**Intuition:** A building is capped by the smaller of its row and column maxima to preserve both silhouettes.

```

class Solution {
    public int maxIncreaseKeepingSkyline(int[][] grid) {
        int n = grid.length;
        int[] rowMax = new int[n];
        int[] colMax = new int[n];
        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) {
                rowMax[r] = Math.max(rowMax[r], grid[r][c]);
                colMax[c] = Math.max(colMax[c], grid[r][c]);
            }
        }
        int total = 0;
        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) {
                total += Math.min(rowMax[r], colMax[c]) - grid[r][c];
            }
        }
        return total;
    }
}

```

**Time**  $O(n^2)$  | **Space**  $O(n)$

## #825 Friends Of Appropriate Ages

**Description:** Count friend requests; x friends y unless `age[y] <= 0.5*age[x]+7` , `age[y] > age[x]` , or `age[y]>100 && age[x]<100` .

**Key:** bucket by age (1..120); for each pair of ages count valid requests.

**Intuition:** Ages are bounded, so counting per age-bucket pair collapses the problem to constant-size work.

```

class Solution {
    public int numFriendRequests(int[] ages) {
        int[] count = new int[121];
        for (int a : ages) {
            count[a]++;
        }
        int result = 0;
        for (int x = 1; x <= 120; x++) {
            for (int y = 1; y <= 120; y++) {
                if (y <= 0.5 * x + 7 || y > x || (y > 100 && x < 100)) {
                    continue;
                }
                result += count[x] * count[y];
                if (x == y) {
                    result -= count[x];
                }
            }
        }
        return result;
    }
}

```

**Time**  $O(n + 120^2)$  | **Space**  $O(1)$

## #838 Push Dominoes

**Description:** Simulate falling dominoes given a string of 'L', 'R', '.'.

**Key:** compute net force at each cell — R pushes right (+, decaying), L pushes left (-, decaying); sign decides final state.

**Intuition:** Each cell's outcome is the balance of forces from the nearest pushers on either side.

```
class Solution {
    public String pushDominoes(String dominoes) {
        int n = dominoes.length();
        int[] force = new int[n];
        int f = 0;
        for (int i = 0; i < n; i++) {
            char ch = dominoes.charAt(i);
            if (ch == 'R') {
                f = n;
            } else if (ch == 'L') {
                f = 0;
            } else {
                f = Math.max(f - 1, 0);
            }
            force[i] += f;
        }
        f = 0;
        for (int i = n - 1; i >= 0; i--) {
            char ch = dominoes.charAt(i);
            if (ch == 'L') {
                f = n;
            } else if (ch == 'R') {
                f = 0;
            } else {
                f = Math.max(f - 1, 0);
            }
            force[i] -= f;
        }
        StringBuilder builder = new StringBuilder();
        for (int x : force) {
            builder.append(x > 0 ? 'R' : x < 0 ? 'L' : '.');
        }
        return builder.toString();
    }
}
```

**Time**  $O(n)$  | **Space**  $O(n)$

## #845 Longest Mountain in Array

**Description:** Find the length of the longest mountain (strictly up then strictly down).

**Key:** find each up-slope start, extend up then down; a valid mountain needs both slopes.

**Intuition:** Scan to a peak by rising, then descend; the span counts only when both directions occurred.

```

class Solution {
    public int longestMountain(int[] arr) {
        int n = arr.length, best = 0, i = 0;
        while (i < n) {
            int start = i;
            if (i + 1 < n && arr[i] < arr[i + 1]) {
                while (i + 1 < n && arr[i] < arr[i + 1]) {
                    i++;
                }
                if (i + 1 < n && arr[i] > arr[i + 1]) {
                    while (i + 1 < n && arr[i] > arr[i + 1]) {
                        i++;
                    }
                    best = Math.max(best, i - start + 1);
                }
            }
            if (i == start) {
                i++;
            }
        }
        return best;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #896 Monotonic Array

**Description:** Return true if the array is entirely non-increasing or entirely non-decreasing.

**Key:** track whether any increase and any decrease occurred; monotonic iff not both.

**Intuition:** A single pass noting direction changes detects non-monotonicity.

```

class Solution {
    public boolean isMonotonic(int[] nums) {
        boolean inc = false, dec = false;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > nums[i - 1]) {
                inc = true;
            } else if (nums[i] < nums[i - 1]) {
                dec = true;
            }
        }
        return !(inc && dec);
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #912 Sort an Array

**Description:** Sort an integer array. Implement an  $O(n \log n)$  sort.

**Key:** merge sort with a reusable buffer.

**Intuition:** Recursively split, sort halves, and merge — stable  $O(n \log n)$  without library sort.

```

class Solution {
    public int[] sortArray(int[] nums) {
        mergeSort(nums, 0, nums.length - 1, new int[nums.length]);
        return nums;
    }
    private void mergeSort(int[] nums, int lo, int hi, int[] buf) {
        if (lo >= hi) {
            return;
        }
        int mid = (lo + hi) >> 1;
        mergeSort(nums, lo, mid, buf);
        mergeSort(nums, mid + 1, hi, buf);
        int i = lo, j = mid + 1, k = lo;
        while (i <= mid && j <= hi) {
            if (nums[i] <= nums[j]) {
                buf[k++] = nums[i++];
            } else {
                buf[k++] = nums[j++];
            }
        }
        while (i <= mid) {
            buf[k++] = nums[i++];
        }
        while (j <= hi) {
            buf[k++] = nums[j++];
        }
        System.arraycopy(buf, lo, nums, lo, hi - lo + 1);
    }
}

```

**Time**  $O(n \log n)$  | **Space**  $O(n)$

## #974 Subarray Sums Divisible by K

**Description:** Count subarrays whose sum is divisible by k.

**Key:** prefix-sum remainder counts; subarrays between equal remainders are divisible.

**Intuition:** Two prefixes with the same remainder mod k bound a divisible subarray; count pairs per remainder.

```

class Solution {
    public int subarraysDivByK(int[] nums, int k) {
        int[] count = new int[k];
        count[0] = 1;
        int sum = 0, result = 0;
        for (int x : nums) {
            sum += x;
            int rem = ((sum % k) + k) % k;
            result += count[rem];
            count[rem]++;
        }
        return result;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(k)$

## #977 Squares of a Sorted Array

**Description:** Return the squares of a sorted array in sorted order.

**Key:** opposite pointers; the larger absolute value is at one of the two ends — write from the back.

**Intuition:** The biggest squares sit at the extremes, so fill the result from the largest slot inward.

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        int i = 0, j = n - 1, k = n - 1;
        while (i <= j) {
            int sqI = nums[i] * nums[i], sqJ = nums[j] * nums[j];
            if (sqI > sqJ) {
                result[k--] = sqI;
                i++;
            } else {
                result[k--] = sqJ;
                j--;
            }
        }
        return result;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$  excluding output

## #1013 Partition Array Into Three Parts With Equal Sum

**Description:** Can the array be split into three contiguous parts with equal sums?

**Key:** total must be divisible by 3; sweep accumulating, counting completed `total/3` parts.

**Intuition:** Greedily close off a part each time the running sum hits one-third; succeed if at least three parts form.

```
class Solution {
    public boolean canThreePartsEqualSum(int[] arr) {
        int total = 0;
        for (int x : arr) {
            total += x;
        }
        if (total % 3 != 0) {
            return false;
        }
        int target = total / 3, running = 0, parts = 0;
        for (int i = 0; i < arr.length; i++) {
            running += arr[i];
            if (running == target) {
                parts++;
                running = 0;
                if (parts == 2 && i < arr.length - 1) {
                    return true;
                }
            }
        }
        return false;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## #1074 Number of Submatrices That Sum to Target

**Description:** Count submatrices summing to target.

**Key:** fix a pair of rows, collapse columns to a 1D array, then apply prefix-sum hashmap (#560) per column.

**Intuition:** Reduce 2D to 1D by fixing the top and bottom rows, then count target-sum subarrays.

```
class Solution {
    public int numSubmatrixSumTarget(int[][] matrix, int target) {
        int m = matrix.length, n = matrix[0].length;
        int[][] prefix = new int[m + 1][n];
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                prefix[r + 1][c] = prefix[r][c] + matrix[r][c];
            }
        }
        int result = 0;
        for (int top = 0; top < m; top++) {
            for (int bottom = top + 1; bottom <= m; bottom++) {
                Map<Integer, Integer> count = new HashMap<>();
                count.put(0, 1);
                int sum = 0;
                for (int c = 0; c < n; c++) {
                    sum += prefix[bottom][c] - prefix[top][c];
                    result += count.getOrDefault(sum - target, 0);
                    count.merge(sum, 1, Integer::sum);
                }
            }
        }
        return result;
    }
}
```

**Time**  $O(m^2 * n)$  | **Space**  $O(n)$

## #1109 Corporate Flight Bookings

**Description:** Given bookings `[first, last, seats]`, return total seats booked per flight.

**Key:** difference array — add seats at `first`, subtract at `last+1`, prefix-sum.

**Intuition:** Range increments become two endpoint marks resolved by one prefix pass.

```
class Solution {
    public int[] corpFlightBookings(int[][] bookings, int n) {
        int[] diff = new int[n + 1];
        for (int[] b : bookings) {
            diff[b[0] - 1] += b[2];
            diff[b[1]] -= b[2];
        }
        int[] result = new int[n];
        int running = 0;
        for (int i = 0; i < n; i++) {
            running += diff[i];
            result[i] = running;
        }
        return result;
    }
}
```

**Time**  $O(n + \text{bookings})$  | **Space**  $O(n)$

## #1213 Intersection of Three Sorted Arrays

**Description:** Return the common elements of three sorted arrays.

**Key:** three pointers; advance the smallest; record when all three match.

**Intuition:** Like merging — advance whichever pointer lags so all three meet at shared values.

```
class Solution {
    public List<Integer> arraysIntersection(int[] arr1, int[] arr2, int[] arr3) {
        List<Integer> result = new ArrayList<>();
        int i = 0, j = 0, k = 0;
        while (i < arr1.length && j < arr2.length && k < arr3.length) {
            if (arr1[i] == arr2[j] && arr2[j] == arr3[k]) {
                result.add(arr1[i]);
                i++;
                j++;
                k++;
            } else if (arr1[i] <= arr2[j] && arr1[i] <= arr3[k]) {
                i++;
            } else if (arr2[j] <= arr1[i] && arr2[j] <= arr3[k]) {
                j++;
            } else {
                k++;
            }
        }
        return result;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$  excluding output

## #1275 Find Winner on a Tic Tac Toe Game

**Description:** Given alternating A/B moves, determine the winner, "Draw", or "Pending".

**Key:** tally row/col/diagonal sums with +1 for A and -1 for B;  $|\text{sum}| == 3$  wins.

**Intuition:** Signed counts per line reveal a win the moment any line reaches +3 or -3.



```

class Solution {
    public String tictactoe(int[][] moves) {
        int[] rows = new int[3];
        int[] cols = new int[3];
        int diag = 0, anti = 0;
        for (int t = 0; t < moves.length; t++) {
            int sign = (t % 2 == 0) ? 1 : -1;
            int r = moves[t][0], c = moves[t][1];
            rows[r] += sign;
            cols[c] += sign;
            if (r == c) {
                diag += sign;
            }
            if (r + c == 2) {
                anti += sign;
            }
            if (Math.abs(rows[r]) == 3 || Math.abs(cols[c]) == 3
                || Math.abs(diag) == 3 || Math.abs(anti) == 3) {
                return sign == 1 ? "A" : "B";
            }
        }
        return moves.length == 9 ? "Draw" : "Pending";
    }
}

```

**Time**  $O(1)$  | **Space**  $O(1)$

## #1351 Count Negative Numbers in a Sorted Matrix

**Description:** Count negatives in a matrix sorted descending by row and column.

**Key:** staircase walk from bottom-left; on a negative move up, else move right.

**Intuition:** From the bottom-left, sorted order lets each step rule out a whole row or column of negatives.

```

class Solution {
    public int countNegatives(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int r = m - 1, c = 0, count = 0;
        while (r >= 0 && c < n) {
            if (grid[r][c] < 0) {
                count += n - c;
                r--;
            } else {
                c++;
            }
        }
        return count;
    }
}

```

**Time**  $O(m + n)$  | **Space**  $O(1)$

## #1424 Diagonal Traverse II

**Description:** Traverse a jagged 2D list diagonally from bottom-left to top-right.

**Key:** group cells by `r+c`; within a diagonal, larger row comes first (bottom-left start).

**Intuition:** Cells on a diagonal share `r+c`; emitting rows in descending order gives the bottom-left-up direction.

```

class Solution {
    public int[] findDiagonalOrder(List<List<Integer>> nums) {
        Map<Integer, List<Integer>> diagonals = new HashMap<>();
        int total = 0, maxKey = 0;
        for (int r = 0; r < nums.size(); r++) {
            for (int c = 0; c < nums.get(r).size(); c++) {
                int key = r + c;
                diagonals.computeIfAbsent(key, x -> new ArrayList<>()).add(nums.get(r).get(c));
                maxKey = Math.max(maxKey, key);
                total++;
            }
        }
        int[] result = new int[total];
        int idx = 0;
        for (int key = 0; key <= maxKey; key++) {
            List<Integer> diag = diagonals.get(key);
            if (diag == null) {
                continue;
            }
            for (int i = diag.size() - 1; i >= 0; i--) {
                result[idx++] = diag.get(i);
            }
        }
        return result;
    }
}

```

**Time**  $O(\text{total})$  | **Space**  $O(\text{total})$

## #1460 Make Two Arrays Equal by Reversing Sub-arrays

**Description:** Can target become arr after reversing any sub-arrays any number of times?

**Key:** reversals allow arbitrary reordering, so the arrays just need identical multisets.

**Intuition:** Any permutation is reachable through reversals, so only element counts matter.

```

class Solution {
    public boolean canBeEqual(int[] target, int[] arr) {
        int[] count = new int[1001];
        for (int x : target) {
            count[x]++;
        }
        for (int x : arr) {
            count[x]--;
        }
        for (int c : count) {
            if (c != 0) {
                return false;
            }
        }
        return true;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #1480 Running Sum of 1d Array

**Description:** Return the running (prefix) sum of the array.

**Key:** accumulate in-place, each element adds the previous.

**Intuition:** Each running sum is the prior running sum plus the current value.

```
class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] += nums[i - 1];
        }
        return nums;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

## #1498 Number of Subsequences That Satisfy the Given Sum Condition

**Description:** Count subsequences where  $\min + \max \leq \text{target}$ . Answer mod  $1e9+7$ .

**Key:** sort; opposite pointers — if  $\text{nums}[i] + \text{nums}[j] \leq \text{target}$ , all subsets of  $(i, j]$  with  $i$  included count  $\rightarrow 2^{(j-i)}$ .

**Intuition:** After sorting, fixing the minimum lets every subset of the elements up to the max be chosen freely.

```
class Solution {
    public int numSubseq(int[] nums, int target) {
        int MOD = 1_000_000_007, n = nums.length;
        Arrays.sort(nums);
        long[] pow = new long[n];
        pow[0] = 1;
        for (int i = 1; i < n; i++) {
            pow[i] = pow[i - 1] * 2 % MOD;
        }
        int i = 0, j = n - 1;
        long result = 0;
        while (i <= j) {
            if (nums[i] + nums[j] <= target) {
                result = (result + pow[j - i]) % MOD;
                i++;
            } else {
                j--;
            }
        }
        return (int) result;
    }
}
```

**Time**  $O(n \log n)$  | **Space**  $O(n)$

## #1748 Sum of Unique Elements

**Description:** Sum all elements that appear exactly once.

**Key:** frequency count; add values with count == 1.

**Intuition:** Count occurrences, then sum only the singletons.

```

class Solution {
    public int sumOfUnique(int[] nums) {
        int[] count = new int[101];
        for (int x : nums) {
            count[x]++;
        }
        int sum = 0;
        for (int v = 1; v <= 100; v++) {
            if (count[v] == 1) {
                sum += v;
            }
        }
        return sum;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #1762 Buildings With an Ocean View

**Description:** Return indices of buildings with nothing taller to their right (ocean to the right).

**Key:** scan right-to-left tracking max height; a building qualifies if taller than all seen so far.

**Intuition:** Sweeping from the ocean side, only record buildings that exceed every taller one already passed.

```

class Solution {
    public int[] findBuildings(int[] heights) {
        List<Integer> result = new ArrayList<>();
        int max = 0;
        for (int i = heights.length - 1; i >= 0; i--) {
            if (heights[i] > max) {
                result.add(i);
                max = heights[i];
            }
        }
        int[] arr = new int[result.size()];
        for (int i = 0; i < arr.length; i++) {
            arr[i] = result.get(arr.length - 1 - i);
        }
        return arr;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$  excluding output

## #1868 Product of Two Run-Length Encoded Arrays

**Description:** Multiply two run-length-encoded arrays element-wise; return the RLE product.

**Key:** two pointers over the runs; multiply values, take the min remaining length, decrement, merge equal adjacent products.

**Intuition:** March through both run lists together, consuming the shorter overlap and coalescing equal results.

```

class Solution {
    public List<List<Integer>> findRLEArray(int[][] encoded1, int[][] encoded2) {
        List<List<Integer>> result = new ArrayList<>();
        int i = 0, j = 0;
        while (i < encoded1.length && j < encoded2.length) {
            int product = encoded1[i][0] * encoded2[j][0];
            int len = Math.min(encoded1[i][1], encoded2[j][1]);
            if (!result.isEmpty() && result.get(result.size() - 1).get(0) == product) {
                List<Integer> last = result.get(result.size() - 1);
                last.set(1, last.get(1) + len);
            } else {
                result.add(new ArrayList<>(Arrays.asList(product, len)));
            }
            encoded1[i][1] -= len;
            encoded2[j][1] -= len;
            if (encoded1[i][1] == 0) {
                i++;
            }
            if (encoded2[j][1] == 0) {
                j++;
            }
        }
        return result;
    }
}

```

**Time**  $O(n + m)$  | **Space**  $O(1)$  excluding output

## #1877 Minimize Maximum Pair Sum in Array

**Description:** Pair up elements to minimize the maximum pair sum.

**Key:** sort, pair smallest with largest via opposite pointers; track the max pair sum.

**Intuition:** Pairing extremes balances the sums, keeping the largest pair as small as possible.

```

class Solution {
    public int minPairSum(int[] nums) {
        Arrays.sort(nums);
        int i = 0, j = nums.length - 1, max = 0;
        while (i < j) {
            max = Math.max(max, nums[i] + nums[j]);
            i++;
            j--;
        }
        return max;
    }
}

```

**Time**  $O(n \log n)$  | **Space**  $O(1)$

## #1893 Check if All the Integers in a Range Are Covered

**Description:** Are all integers in  $[left, right]$  covered by at least one given interval?

**Key:** difference array over the small value domain (1..50); prefix-sum to test coverage.

**Intuition:** Mark interval starts and ends, then a prefix sweep shows which points are covered.

```

class Solution {
    public boolean isCovered(int[][] ranges, int left, int right) {
        int[] diff = new int[52];
        for (int[] r : ranges) {
            diff[r[0]]++;
            diff[r[1] + 1]--;
        }
        int running = 0;
        for (int i = 1; i <= 50; i++) {
            running += diff[i];
            if (i >= left && i <= right && running <= 0) {
                return false;
            }
        }
        return true;
    }
}

```

**Time**  $O(n + 50)$  | **Space**  $O(1)$

## #1894 Find the Student that Will Replace the Chalk

**Description:** Students consume chalk in order cyclically; find who runs out.

**Key:** reduce  $k$  modulo the total sum, then sweep once.

**Intuition:** Whole cycles repeat, so only the remainder after a full round determines the failing student.

```

class Solution {
    public int chalkReplacer(int[] chalk, int k) {
        long total = 0;
        for (int c : chalk) {
            total += c;
        }
        k = (int) (k % total);
        for (int i = 0; i < chalk.length; i++) {
            if (k < chalk[i]) {
                return i;
            }
            k -= chalk[i];
        }
        return 0;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #1920 Build Array from Permutation

**Description:** Return `result[i] = nums[nums[i]]`.

**Key:** straightforward indexed build ( $O(n)$  extra space) or encode two values per slot for  $O(1)$  space.

**Intuition:** Each output is a double lookup into the permutation.

```
class Solution {
    public int[] buildArray(int[] nums) {
        int[] result = new int[nums.length];
        for (int i = 0; i < nums.length; i++) {
            result[i] = nums[nums[i]];
        }
        return result;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(n)$  excluding output

---

## #1929 Concatenation of Array

---

**Description:** Return `nums` concatenated with itself.

**Key:** allocate  $2n$  and copy `nums` into both halves.

**Intuition:** Copy each element into position `i` and `i+n`.

```
class Solution {
    public int[] getConcatenation(int[] nums) {
        int n = nums.length;
        int[] result = new int[2 * n];
        for (int i = 0; i < n; i++) {
            result[i] = nums[i];
            result[i + n] = nums[i];
        }
        return result;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$  excluding output